



## University of Chittagong

Department of Computer Science and Engineering

---

# Machine Learning Lab Report

Five Interactive Machine Learning Problems

**Course Code: CSE 816**

---

### Submitted To

Prof. Dr. Mohammad Osiur Rahman  
Department of Computer Science and  
Engineering  
University of Chittagong

### Submitted By

Md. Kais  
Student ID: 20701003  
Department of Computer Science and  
Engineering  
University of Chittagong

**Submission Date: September 14, 2026**

---

# Declaration and Scope

---

This Laboratory Report was prepared for Prof. Dr. Mohammad Osiur Rahman at University of Chittagong from the current source code, datasets, tests, `README.md` files, and project reports in the workspace. It describes the actual implementation rather than an imagined library version. The fixed header shows CSE 816 on the left and 20701003 on the right on every page; every footer contains only the centered page number.

The applications are educational. The BMI, diabetes, and cardiac-triage projects must not be used to make health decisions. The customer dataset is fictional, and uploaded CNN images are inference inputs rather than new training data.

## Algorithms and problems covered

| Algorithm     | Problem title                                                                      | Architecture                     |
|---------------|------------------------------------------------------------------------------------|----------------------------------|
| KNN           | Sorting an adult BMI using the five nearest saved examples                         | Python exports; browser predicts |
| Decision Tree | Following readable yes/no rules for synthetic cardiac-triage cards                 | Python trains; browser predicts  |
| Linear SVM    | Visualizing a wide separating margin between two diabetes-screening example groups | Python trains; browser predicts  |
| K-Means       | Grouping fictional customers by age and shopping score                             | Browser trains and predicts      |
| CNN/YOLO      | Finding and drawing boxes around everyday objects in a photo                       | Flask runs pretrained inference  |

---

# Contents

---

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Declaration and Scope</b>                                   | <b>2</b>  |
| <b>1 K-Nearest Neighbors (KNN)</b>                             | <b>9</b>  |
| 1.1 Problem Description . . . . .                              | 9         |
| 1.2 Algorithm Description . . . . .                            | 9         |
| 1.2.1 The neighbor-vote story . . . . .                        | 9         |
| 1.3 Dataset Description . . . . .                              | 10        |
| 1.4 Training . . . . .                                         | 11        |
| 1.5 Website Design and Answer Flow . . . . .                   | 12        |
| 1.6 Comprehensive User Manual . . . . .                        | 12        |
| 1.6.1 Recorded website experiment: 170 cm and 100 kg . . . . . | 13        |
| 1.6.2 Inputs and buttons . . . . .                             | 16        |
| 1.6.3 Outputs . . . . .                                        | 17        |
| 1.6.4 Worked example . . . . .                                 | 17        |
| 1.6.5 Troubleshooting . . . . .                                | 17        |
| 1.7 Conclusion . . . . .                                       | 17        |
| <b>2 Decision Tree</b>                                         | <b>18</b> |
| 2.1 Problem Description . . . . .                              | 18        |
| 2.2 The Algorithm . . . . .                                    | 18        |
| 2.2.1 A tree made from questions . . . . .                     | 18        |
| 2.2.2 How the program chooses a question . . . . .             | 19        |
| 2.3 Dataset Description . . . . .                              | 20        |
| 2.4 Training . . . . .                                         | 21        |
| 2.5 Website . . . . .                                          | 21        |
| 2.6 Comprehensive User Manual . . . . .                        | 21        |

|          |                                                                                     |           |
|----------|-------------------------------------------------------------------------------------|-----------|
| 2.6.1    | Recorded website experiment: the one-question blue route . . . . .                  | 22        |
| 2.6.2    | Start and enter a card . . . . .                                                    | 25        |
| 2.6.3    | Read every output . . . . .                                                         | 25        |
| 2.7      | Conclusion . . . . .                                                                | 26        |
| <b>3</b> | <b>Support Vector Machine</b>                                                       | <b>27</b> |
| 3.1      | Problem Description . . . . .                                                       | 27        |
| 3.2      | The Algorithm . . . . .                                                             | 27        |
| 3.2.1    | A wide road between two groups . . . . .                                            | 27        |
| 3.3      | Dataset Description . . . . .                                                       | 28        |
| 3.4      | Training . . . . .                                                                  | 29        |
| 3.5      | Website . . . . .                                                                   | 29        |
| 3.6      | Comprehensive User Manual . . . . .                                                 | 30        |
| 3.6.1    | Recorded website experiment: a point beyond the upper margin . .                    | 30        |
| 3.7      | Conclusion . . . . .                                                                | 35        |
| <b>4</b> | <b>K-Means Clustering</b>                                                           | <b>36</b> |
| 4.1      | Problem Description . . . . .                                                       | 36        |
| 4.2      | The Algorithm . . . . .                                                             | 36        |
| 4.2.1    | Move two team flags until nobody changes team . . . . .                             | 36        |
| 4.3      | Dataset Description . . . . .                                                       | 38        |
| 4.4      | Training . . . . .                                                                  | 38        |
| 4.5      | Website . . . . .                                                                   | 39        |
| 4.6      | Comprehensive User Manual . . . . .                                                 | 39        |
| 4.6.1    | Recorded website experiment: assign a new customer and replay<br>learning . . . . . | 39        |
| 4.7      | Conclusion . . . . .                                                                | 44        |
| <b>5</b> | <b>CNN and YOLO Object Detection</b>                                                | <b>45</b> |
| 5.1      | Problem Description . . . . .                                                       | 45        |
| 5.2      | The Algorithm . . . . .                                                             | 45        |
| 5.2.1    | From pixels to visual clues . . . . .                                               | 45        |
| 5.2.2    | YOLO detection in one model pass . . . . .                                          | 45        |
| 5.3      | Dataset Description . . . . .                                                       | 47        |
| 5.3.1    | What data the current application uses . . . . .                                    | 47        |

|                                           |                                                                  |           |
|-------------------------------------------|------------------------------------------------------------------|-----------|
| 5.4                                       | Training and Model Use . . . . .                                 | 48        |
| 5.4.1                                     | Run the supplied pretrained detector . . . . .                   | 48        |
| 5.4.2                                     | Optional custom training . . . . .                               | 48        |
| 5.5                                       | Website . . . . .                                                | 49        |
| 5.6                                       | Comprehensive User Manual . . . . .                              | 49        |
| 5.6.1                                     | Recorded website experiment: a busy urban intersection . . . . . | 49        |
| 5.6.2                                     | Choose an input . . . . .                                        | 52        |
| 5.6.3                                     | Run and understand the answer . . . . .                          | 53        |
| 5.7                                       | Conclusion . . . . .                                             | 53        |
| <b>Overall Conclusion</b>                 |                                                                  | <b>54</b> |
| <b>A Verification and Reproduction</b>    |                                                                  | <b>55</b> |
| A.1                                       | Commands verified in this workspace . . . . .                    | 55        |
| A.2                                       | One-page run checklist . . . . .                                 | 55        |
| <b>B Glossary for a First-Time Reader</b> |                                                                  | <b>57</b> |
| <b>References</b>                         |                                                                  | <b>58</b> |

---

## List of Figures

---

|     |                                                                                                                                                                                                                                           |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | KNN training-to-browser flow. . . . .                                                                                                                                                                                                     | 12 |
| 1.2 | Stage 1 - initial KNN page. The demonstration starts at 170 cm and 65 kg; the answer card is still in its ready state and the neighbor chart shows the saved teaching examples. . . . .                                                   | 13 |
| 1.3 | Stage 2 - input prepared. Height is 170 cm and weight is 100 kg. The ready message proves that these values have been entered but not yet submitted. . . . .                                                                              | 14 |
| 1.4 | Stage 3 - KNN output for 170 cm and 100 kg. The page calculates BMI 34.6 and reports the teaching category Obese after the five nearest saved BMI examples vote. This is a synthetic screening demonstration, not a diagnosis. . . . .    | 15 |
| 1.5 | Stage 4 - evidence graph for the same KNN answer. The lime diamond is the entered person, numbered rings 1–5 are the selected neighbors, and dashed lines connect the query to those voters. . . . .                                      | 16 |
| 2.1 | The exact tree exported by the current training script. . . . .                                                                                                                                                                           | 19 |
| 2.2 | Decision-tree training and browser prediction flow. . . . .                                                                                                                                                                               | 21 |
| 2.3 | Stage 1 - initial Decision Tree page. Its default toy card is 155 mmHg and 130 bpm, so the previously rendered example occupies the red high leaf. . . . .                                                                                | 22 |
| 2.4 | Stage 2 - new input prepared but not submitted. The boxes now contain BP 130 mmHg and maximum heart rate 150 bpm, while the earlier red result is still visible. Pressing Follow the tree is therefore a necessary function call. . . . . | 23 |
| 2.5 | Stage 3 - Decision Tree output. For BP 130 mmHg and heart rate 150 bpm, the root question $130 \leq 140$ is Yes, so the page returns Low Risk / Discharge as a toy label and highlights the large blue leaf. . . . .                      | 24 |
| 2.6 | Stage 4 - focused explanation and graph. The decision path contains only the root BP comparison, and the bright outline identifies the exact rectangular leaf reached by the input point. . . . .                                         | 25 |
| 3.1 | SVM training, export, and browser scoring flow. . . . .                                                                                                                                                                                   | 30 |
| 3.2 | Stage 1 - initial SVM page. The default example is 112 mg/dL fasting glucose and BMI 29; the full page already explains the learned line, two margins, support examples, and safety boundary. . . . .                                     | 31 |

|     |                                                                                                                                                                                                                                                       |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.3 | Stage 2 - input prepared. The form now contains 150 mg/dL and BMI 33, but the previous result remains until Screen and plot is pressed. . . . .                                                                                                       | 32 |
| 3.4 | Stage 3 - SVM output for glucose 150 mg/dL and BMI 33. The signed decision score is 2.57, so the page reports Higher screening signal and moves the user marker to the red side beyond the upper margin. . . . .                                      | 33 |
| 3.5 | Stage 4 - focused SVM graph. The green diamond is the submitted input; blue and red points are synthetic training examples, ringed points are support examples, and the central strip shows the follow-up margin. . . . .                             | 34 |
| 4.1 | K-Means page-load training and user classification flow. . . . .                                                                                                                                                                                      | 39 |
| 4.2 | Stage 1 - initial K-Means page. The starting demonstration uses age 34 and shopping score 56 and shows the trained red/blue groups, their centroid stars, and a ready-made red-team result. . . . .                                                   | 40 |
| 4.3 | Stage 2 - input prepared. The controls show age 48 and shopping score 86, while the old red-team answer is still visible because the form has not yet been submitted. . . . .                                                                         | 41 |
| 4.4 | Stage 3 - K-Means output. Age 48 and score 86 are assigned to Blue team: Premium Shopper. The page reports a normalized map distance of 20 to the blue center and 63 to the red center; the smaller distance wins. . . . .                            | 42 |
| 4.5 | Stage 4 - additional training information. During replay round 1, every saved dot joins its nearer star and the centroids move toward their current team averages. This animation changes the displayed learning frame, not the final answer. . . . . | 43 |
| 5.1 | End-to-end Sightline request flow. . . . .                                                                                                                                                                                                            | 49 |
| 5.2 | Stage 1 - initial Sightline page. No photograph is selected, the result field says Waiting for a photo, the count and time are blank, and the threshold is 35%. . . . .                                                                               | 50 |
| 5.3 | Stage 2 - input prepared. The busy street photograph appears in the upload preview with its filename and size. Run detection is enabled, but the output panel is still empty. . . . .                                                                 | 50 |
| 5.4 | Stage 3 - live YOLO output at the 35% threshold. The annotated preview shows colored boxes and labels; the website reports 32 objects across 12 classes and an 88 ms warm-run inference time on this machine. . . . .                                 | 51 |
| 5.5 | Stage 4 - focused result evidence. The result badge, annotated frame, count, time, threshold, class count, and strongest-first confidence list are visible together. . . . .                                                                          | 51 |
| 5.6 | Exact CNN experiment before and after website inference. The input bytes on the left produced the annotated JPEG returned by the Flask API on the right. . . . .                                                                                      | 52 |

---

# List of Tables

---

|     |                                                             |    |
|-----|-------------------------------------------------------------|----|
| 1.1 | KNN implementation map. . . . .                             | 10 |
| 1.2 | KNN CSV columns and verified contents. . . . .              | 11 |
| 1.3 | KNN parameters and their effects. . . . .                   | 12 |
| 1.4 | KNN result meanings. . . . .                                | 17 |
| 2.1 | Decision-tree functions and data flow. . . . .              | 20 |
| 2.2 | Meaning of each cardiac demonstration column. . . . .       | 20 |
| 3.1 | Linear-SVM functions and passed values. . . . .             | 28 |
| 3.2 | Meaning of the SVM dataset columns. . . . .                 | 29 |
| 4.1 | K-Means browser functions. . . . .                          | 37 |
| 4.2 | Fields in each fictional customer object. . . . .           | 38 |
| 5.1 | CNN web application functions and information flow. . . . . | 47 |
| 5.2 | Input image information and its meaning. . . . .            | 48 |
| 5.3 | Observed output for the busy-street CNN experiment. . . . . | 52 |

## Lab 1

# K-Nearest Neighbors (KNN)

## Problem: Adult BMI Category from Five Nearby Examples

### 1.1 Problem Description

The BMI Neighbor project accepts adult height in centimetres and weight in kilograms, calculates Body Mass Index, and lets five saved examples vote for one of four labels: Underweight, Normal, Overweight, or Obese. The purpose is to make instance-based classification inspectable. Cover and Hart established the classic nearest-neighbor classification framework [1].

The categories in the synthetic CSV are organized around CDC adult BMI ranges: below 18.5, 18.5–24.9, 25.0–29.9, and at least 30.0 [2]. The page is restricted to an adult teaching context because children and teenagers require age- and sex-specific interpretation.

**Safety boundary:** BMI is a screening measurement, not a diagnosis or a complete description of health. This small synthetic dataset is for algorithm learning; personal medical questions belong with a qualified health professional.

### 1.2 Algorithm Description

#### 1.2.1 The neighbor-vote story

Imagine placing labeled people on a long BMI number line. A new person stands at one BMI value. The computer measures every gap, invites the closest five people, and counts their labels. The most common label wins. If two labels receive the same number of votes, the tied group with the smaller total gap wins.

BMI is computed as

$$\text{BMI} = \frac{\text{weight in kg}}{(\text{height in metres})^2}. \quad (1.1)$$

For 170 cm and 65 kg, BMI is  $65/(1.70)^2 \approx 22.49$ . The one-dimensional distance from a saved example is

$$d_i = |\text{BMI}_{\text{user}} - \text{BMI}_i|. \quad (1.2)$$

```

1 distances = [(abs(example.bmi - bmi), example)
2               for example in self.examples]
3 distances.sort(key=lambda item: item[0])
4 neighbors = distances[: self.k]
```

Listing 1.1: Distance, sorting, and selection in `knn_model.py`.

Table 1.1: KNN implementation map.

| Function / structure                  | Input / return                     | Role and calling flow                                                                      |
|---------------------------------------|------------------------------------|--------------------------------------------------------------------------------------------|
| <code>BmiExample</code>               | Four immutable fields              | Dataclass holding height, weight, calculated BMI, and known label.                         |
| <code>calculate_bmi(h,w)</code>       | cm, kg; BMI float                  | Validates positive values, converts cm to m, and applies the formula.                      |
| <code>load_dataset(path)</code>       | CSV path; example list             | Parses rows, verifies labels, and calculates BMI rather than trusting a stored BMI column. |
| <code>SimpleKNN(k)</code>             | Odd positive integer; model        | Stores neighbor count; current value is 5.                                                 |
| <code>fit(examples)</code>            | Example list; no value             | Training is memory: it copies the labeled examples.                                        |
| <code>nearest_neighbors(bmi)</code>   | BMI; list of $(d, \text{example})$ | Measures, sorts, and returns exactly $k$ records.                                          |
| <code>predict_bmi(bmi)</code>         | BMI; label                         | Counts votes and uses total distance for a tie.                                            |
| <code>predict(h,w)</code>             | cm, kg; dictionary                 | Calls BMI, neighbor, and vote functions; returns the label and evidence.                   |
| <code>export_for_browser(path)</code> | Output path                        | Writes $k$ and all remembered rows to <code>model.json</code> .                            |
| <code>predictWithKnn</code>           | BMI and JSON model; object         | Browser twin of distance, sorting, voting, and tie-breaking.                               |

KNN is called a lazy learner because `fit` does not compress the data into weights. The exported model is the memory itself: a JSON object containing `k`, `feature`, and an array of training-example objects.

## 1.3 Dataset Description

The file `KNN/data/bmi_data.csv` contains 60 deterministic, synthetic rows. It was built by choosing five heights (150, 160, 170, 180, and 190 cm) and twelve target BMI positions around the official adult category boundaries. Weight was calculated using  $\text{weight} = \text{BMI} \times (\text{height in metres})^2$  and rounded to one decimal place. Labels follow the CDC ranges [2]. This creates examples on both sides of important boundaries without using patient records.

Table 1.2: KNN CSV columns and verified contents.

| Column         | Observed range          | Meaning                                                                               |
|----------------|-------------------------|---------------------------------------------------------------------------------------|
| height_cm      | 150–190                 | Synthetic adult height in centimetres; used to calculate BMI and draw the map.        |
| weight_kg      | 37.1–151.6              | Synthetic weight in kilograms; used to calculate BMI and draw the map.                |
| category       | Four words              | Known teaching label. Counts: 10 Underweight, 15 Normal, 15 Overweight, and 20 Obese. |
| Calculated bmi | approximately 16.5–42.0 | Derived at load time, then included in the exported JSON.                             |

The dataset is deliberately small and regular. This helps a learner see the neighbors, but it does not provide demographic diversity or an independent clinical validation sample.

## 1.4 Training

```

1 cd C:\Users\User\machine_learning\KNN
2 python train_model.py
3 python start_app.py

```

Listing 1.2: Train, export, and serve the KNN project.

The training script loads 60 rows, places every fifth row into a 12-row test pile, lets a temporary model remember the remaining 48 rows, and measures held-out accuracy. The verified result was 100% on those 12 synthetic rows. It then fits a final  $k = 5$  model on all 60 rows and writes `dist/model.json`. The browser loads that JSON and repeats the same distance-and-vote rule locally.

Table 1.3: KNN parameters and their effects.

| Choice           | Current         | What changes                                                                                                                                                      |
|------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $k$              | 5               | $k = 3$ follows very local examples and can react to noise; $k = 7$ smooths the vote but may blur a narrow category. Use a positive odd value and retrain/export. |
| Distance feature | BMI only        | Height and weight appear on the graph, but neighbors are chosen by BMI gap. A two-feature distance would be a different model and would require scaling.          |
| Tie rule         | Total gap       | If vote counts tie, the category whose voters have the smaller combined BMI distance wins.                                                                        |
| Dataset rows     | 60              | Added rows can change which five examples are nearest. Keep labels consistent, then rerun <code>python train_model.py</code> .                                    |
| Split rule       | Every fifth row | This deterministic check is reproducible but not a substitute for repeated or external validation.                                                                |

## 1.5 Website Design and Answer Flow

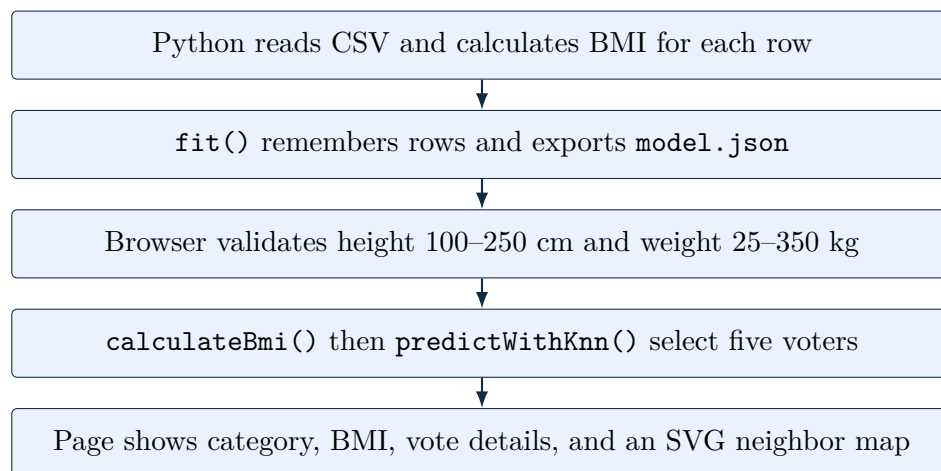


Figure 1.1: KNN training-to-browser flow.

Prediction is entirely client-side after the static files load. The SVG uses height on the  $x$  axis and weight on the  $y$  axis, but the five ringed points are chosen by BMI distance. Dashed lines are visual links, not the metric used by the classifier.

## 1.6 Comprehensive User Manual

## 1.6.1 Recorded website experiment: 170 cm and 100 kg

Figures 1.2 through 1.5 record the complete interaction, from the untouched page to the five-neighbor evidence.

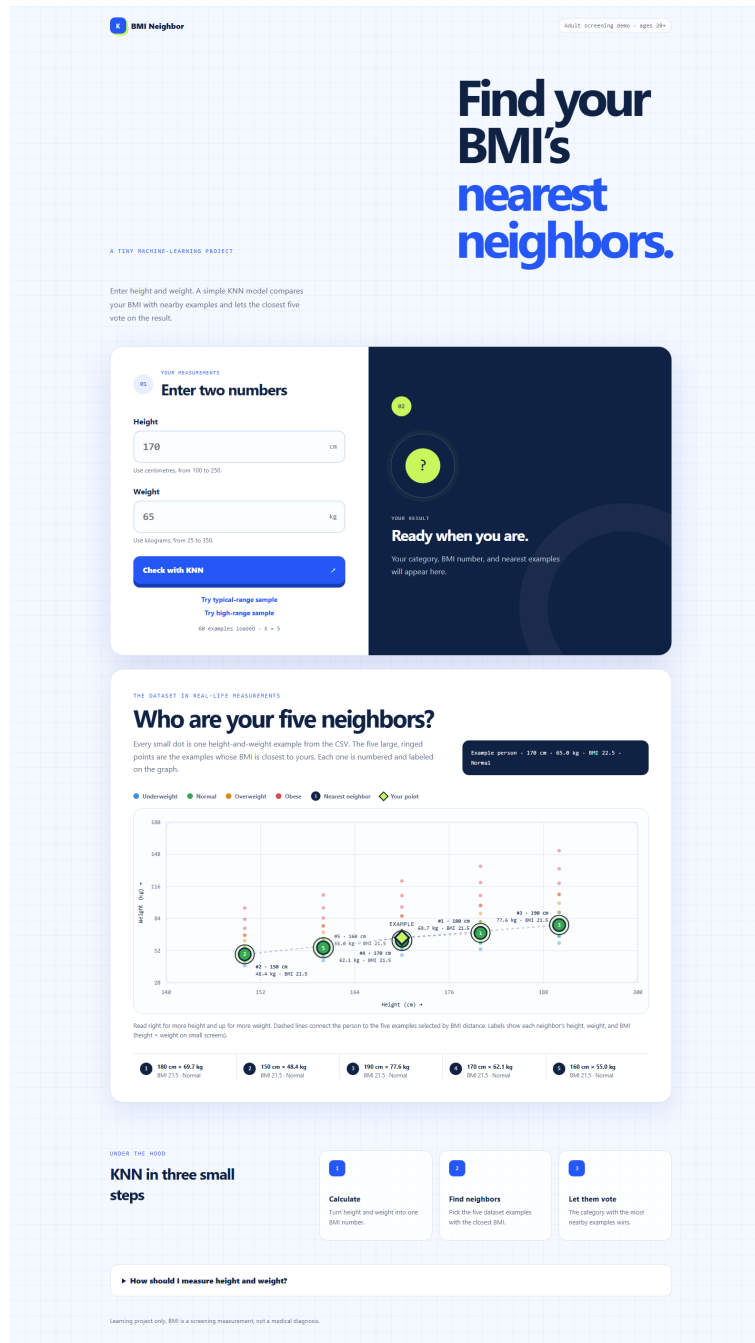


Figure 1.2: Stage 1 - initial KNN page. The demonstration starts at 170 cm and 65 kg; the answer card is still in its ready state and the neighbor chart shows the saved teaching examples.

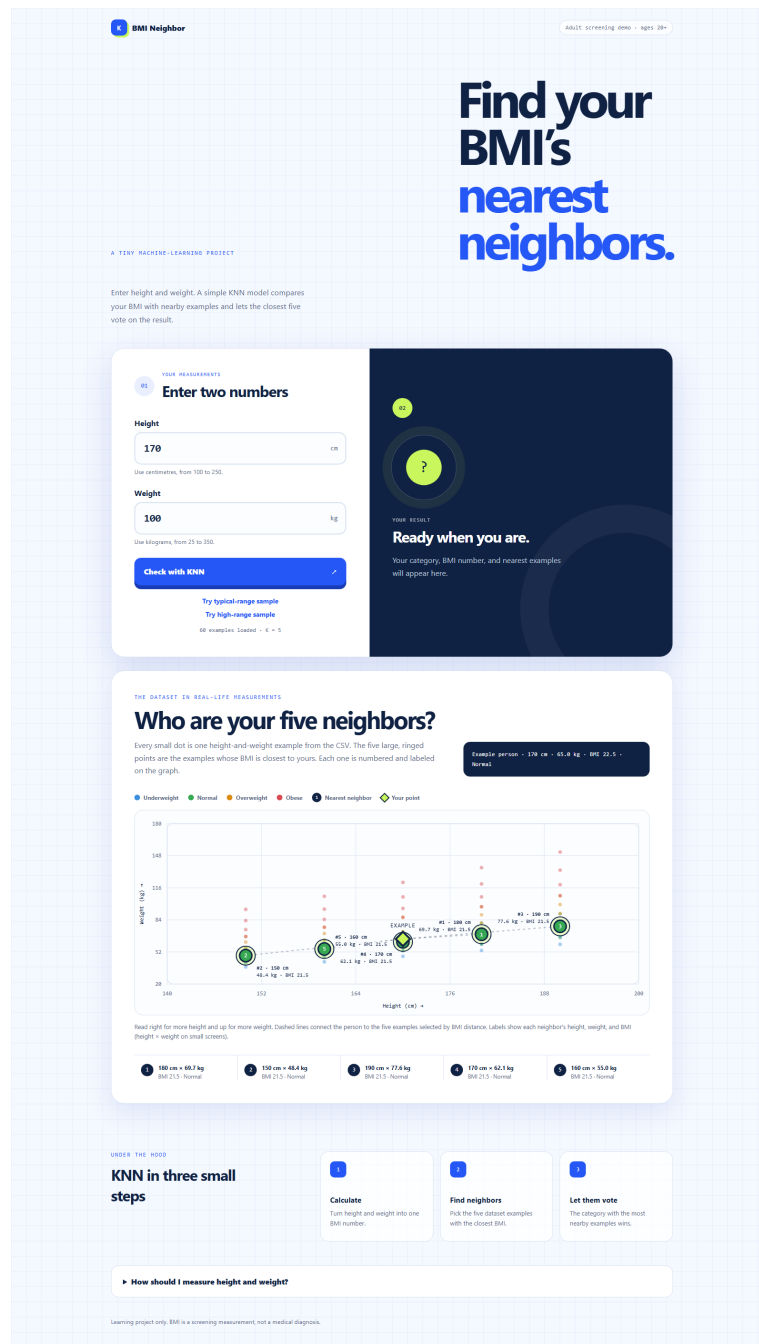


Figure 1.3: Stage 2 - input prepared. Height is 170 cm and weight is 100 kg. The ready message proves that these values have been entered but not yet submitted.

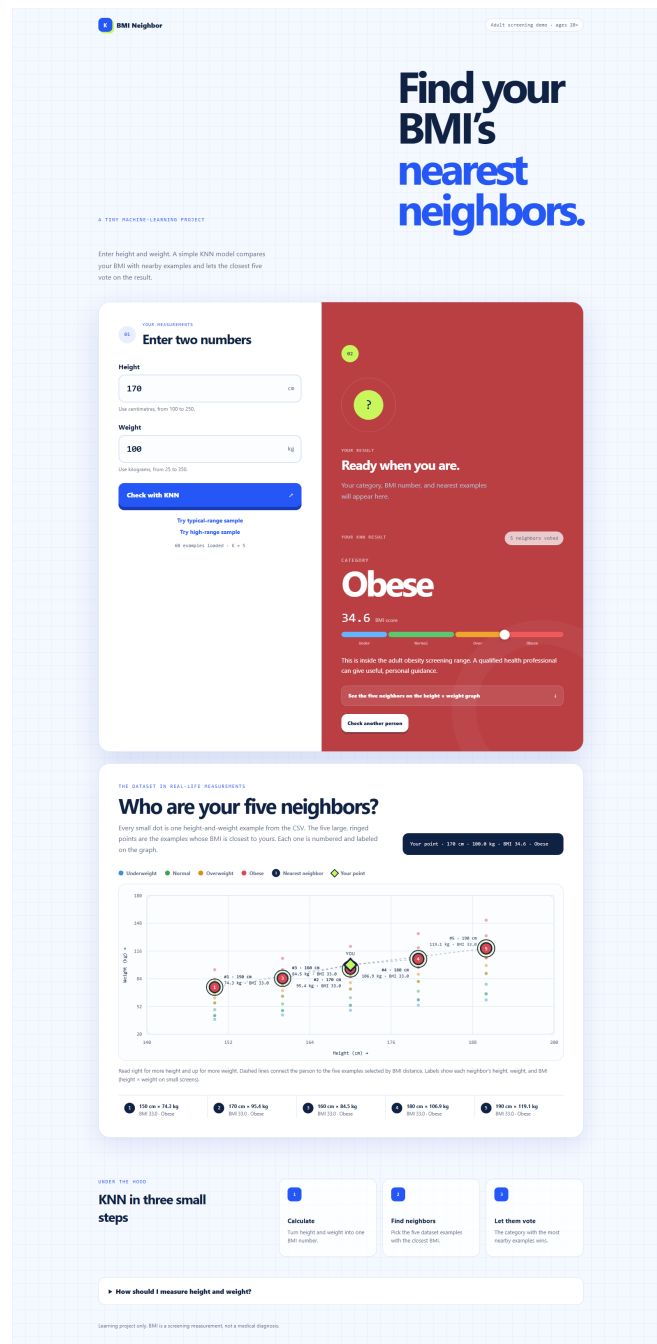


Figure 1.4: Stage 3 - KNN output for 170 cm and 100 kg. The page calculates BMI 34.6 and reports the teaching category Obese after the five nearest saved BMI examples vote. This is a synthetic screening demonstration, not a diagnosis.



Figure 1.5: Stage 4 - evidence graph for the same KNN answer. The lime diamond is the entered person, numbered rings 1–5 are the selected neighbors, and dashed lines connect the query to those voters.

The observable input-output record is therefore

$$(170 \text{ cm}, 100 \text{ kg}) \longrightarrow \text{BMI } 34.6 \longrightarrow \text{Obese teaching label.}$$

The output image also lists all five voters, making the prediction traceable instead of presenting only a final word.

## 1.6.2 Inputs and buttons

1. Enter adult height from 100 to 250 cm. Enter the number measured from the floor to the top of the head, without shoes when possible.
2. Enter weight from 25 to 350 kg. Use kilograms, not pounds.
3. Optionally press the 170 cm/65 kg or 170 cm/100 kg example button. Presets fill the same form and do not bypass validation.
4. Press **Check with KNN**. The button remains disabled until the JSON model is loaded.
5. Use **Reset** to restore the initial example and clear the result.

### 1.6.3 Outputs

The result card reports the calculated BMI to one decimal place and one winning category. The category colors are reading aids only. The neighbor panel lists five voters with rank, height, weight, BMI, label, and distance. On the graph, the lime diamond is the entered person; small colored dots are all 60 saved examples; large numbered rings are the five voters.

Table 1.4: KNN result meanings.

| Displayed item     | Meaning                                                                                                                      |
|--------------------|------------------------------------------------------------------------------------------------------------------------------|
| Underweight / blue | Five nearby synthetic BMI examples voted for the below-18.5 teaching category.                                               |
| Normal / green     | The vote selected the 18.5–24.9 teaching category. The CDC uses the term Healthy Weight; the code’s literal label is Normal. |
| Overweight / amber | The vote selected the 25.0–29.9 teaching category.                                                                           |
| Obese / red        | The vote selected BMI at least 30; it is not a diagnosis or statement about the whole person.                                |
| BMI number         | A height/weight ratio, not a percentage.                                                                                     |
| Distance           | Absolute difference between two BMI values. Smaller means nearer.                                                            |
| Neighbor rank      | 1 is closest; 5 is the last voter included.                                                                                  |

### 1.6.4 Worked example

For 170 cm and 65 kg, the page calculates  $\text{BMI} \approx 22.5$ . It compares 22.5 with all 60 saved BMI values, sorts the absolute differences, and lets the first five labels vote. The expected teaching label is Normal. A user should report this as “the five nearest synthetic examples voted for Normal,” not “the person is medically normal.”

### 1.6.5 Troubleshooting

If the button stays disabled, wait for the model-ready state and confirm the page is served through `start_app.py`; opening `index.html` directly can block `fetch('model.json')`. If a field is red, check its unit and range. If a visually distant graph point becomes a neighbor, remember that KNN uses BMI, so different height/weight pairs can share almost the same BMI. After editing the CSV, always rerun `python train_model.py`.

## 1.7 Conclusion

This project exposes KNN’s defining idea: training stores examples and prediction performs the real work. The visible neighbor evidence makes the answer auditable, while the synthetic and one-feature design makes the limits clear.

## Lab 2

---

# Decision Tree

---

## Problem: Explainable Cardiac-Triage Demonstration

### 2.1 Problem Description

This laboratory asks a deliberately small question: can a computer turn two numbers on a synthetic triage card into a visible chain of yes/no decisions? The two inputs are systolic blood pressure (BP), measured in mmHg, and maximum heart rate (HR), measured in beats per minute. The output is the teaching label **low** or **high**. The useful part is not a medical verdict; it is the ability to see every rule that the program followed.

**Safety boundary:** The 48 records and their labels are synthetic. Real chest-pain and cardiac assessment uses symptoms, history, examination, ECG, repeated blood tests such as troponin, and sometimes imaging. A two-input classroom tree must never guide treatment or delay emergency care [3, 4].

### 2.2 The Algorithm

#### 2.2.1 A tree made from questions

A decision tree resembles the game “Twenty Questions.” At an internal node, the program asks a question such as “Is BP at most 140?” A yes answer follows the left branch; a no answer follows the right branch. A leaf is an ending box that stores the class to return. This project’s complete learned tree is:

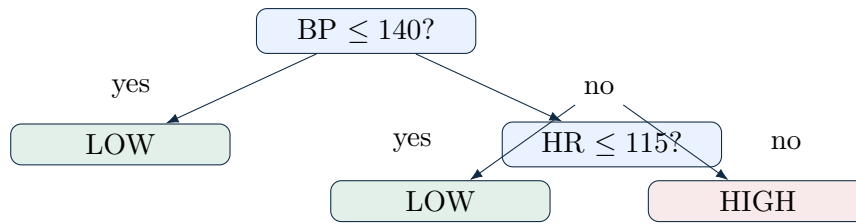


Figure 2.1: The exact tree exported by the current training script.

**Plain meaning.** For BP 155 and HR 130, the first answer is no and the second is also no, so the route ends at HIGH. A prediction is simply a walk from the top box to one ending box.

## 2.2.2 How the program chooses a question

The CART method repeatedly looks for a split that makes the child groups purer than their parent [5]. For a node with class proportions  $p_c$ , its Gini impurity is

$$G = 1 - \sum_c p_c^2. \quad (2.1)$$

A box containing only one label has  $G = 0$ . A mixed half-low, half-high box has  $G = 1 - (0.5^2 + 0.5^2) = 0.5$ . For each feature, the implementation sorts its unique values and tests the midpoint between every neighboring pair. A candidate's score is the size-weighted impurity of its two children:

$$G_{\text{split}} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R. \quad (2.2)$$

The candidate with the smallest score becomes the question. The routine then calls itself once for the left subset and once for the right subset. Recursion stops when all labels match, depth reaches `max_depth`, too few records remain, or no useful split exists. The leaf uses the majority label.

```

1 for feature in ("bp", "hr"):
2     values = sorted({row.value(feature) for row in rows})
3     thresholds = [(a + b) / 2 for a, b in zip(values, values[1:])]
4     for threshold in thresholds:
5         left = [r for r in rows if r.value(feature) <= threshold]
6         right = [r for r in rows if r.value(feature) > threshold]
7         score = (len(left) * gini_impurity(left)
8                 + len(right) * gini_impurity(right)) / len(rows)
9         if score < best_score:
10             best = (feature, threshold, left, right)

```

Listing 2.1: Core split search, simplified from `decisionTree/decision_tree_model.py`.

The strict less-than comparison in the last line gives deterministic tie handling: the first equally good candidate remains selected. The row object is a `TrainingExample`; a `TreeNode` holds an ID, sample counts, prediction, optional feature and threshold, plus

left/right child references. Export converts references into nested JSON so JavaScript can read the tree.

Table 2.1: Decision-tree functions and data flow.

| Function                   | Input and parameters         | Output or call                                             |
|----------------------------|------------------------------|------------------------------------------------------------|
| <code>load_dataset</code>  | CSV path                     | List of typed <code>TrainingExample</code> objects         |
| <code>gini_impurity</code> | A list of rows               | One impurity number                                        |
| <code>_best_split</code>   | Rows at the current node     | Best feature, threshold, and two row lists                 |
| <code>_build</code>        | Rows and current depth       | Calls itself for children; returns a <code>TreeNode</code> |
| <code>fit</code>           | All training rows            | Resets node IDs and stores the root                        |
| <code>predict</code>       | One BP and HR pair           | Walks the stored nodes and returns a label                 |
| <code>export</code>        | Fitted model and output path | JSON tree, labels, rules, ranges, and accuracy             |

## 2.3 Dataset Description

The file `decisionTree/data/cardiac_triage.csv` contains 48 teaching cards. It was created as the Cartesian product of eight BP levels (90, 105, 120, 135, 145, 155, 170, 180) and six HR levels (60, 80, 100, 120, 140, 160). This balanced grid makes the two-dimensional rule easy to inspect; it is not a patient sample. There are 36 **low** and 12 **high** labels.

Table 2.2: Meaning of each cardiac demonstration column.

| Column                      | Stored values | Plain-language meaning                                |
|-----------------------------|---------------|-------------------------------------------------------|
| <code>systolic_bp</code>    | 90–180 mmHg   | Pressure when the heart contracts; it is feature one. |
| <code>max_heart_rate</code> | 60–160 bpm    | A synthetic maximum-rate value; it is feature two.    |
| <code>triage_label</code>   | low/high      | The answer supplied to the learner, not a diagnosis.  |

The labels were arranged so only cards with BP above 140 and HR above 115 are **high**. Training rediscovers that design. Authoritative guidance is cited to show why real assessment is much richer, not to claim that the synthetic labeling rule is clinical guidance [3, 4].

## 2.4 Training

From the workspace, run:

```
1 cd decisionTree
2 python train_model.py
3 python start_app.py
```

The first command loads the CSV and creates a `SimpleDecisionTree` with maximum depth 2 and minimum split size 2. It then calls `fit`, measures predictions on all 48 rows, and writes `dist/model.json`. The verified result was 100% training accuracy. That only confirms the tree reproduces this designed grid; there is no held-out clinical validation.

- Raising `max_depth` permits more questions and finer regions, but can memorize noise. Lowering it to 1 removes the HR question.
- Increasing `min_samples_split` refuses to divide small groups, producing a shorter, smoother tree.
- Changing CSV labels moves the best thresholds. Retrain after every dataset edit or the browser will continue using old JSON.

## 2.5 Website

The server only hosts static files; the already-trained JSON is the browser's source of truth. JavaScript validates the node structure, walks the tree, records the visited path, computes colored decision regions, and redraws the SVG without sending the entered numbers to Python.

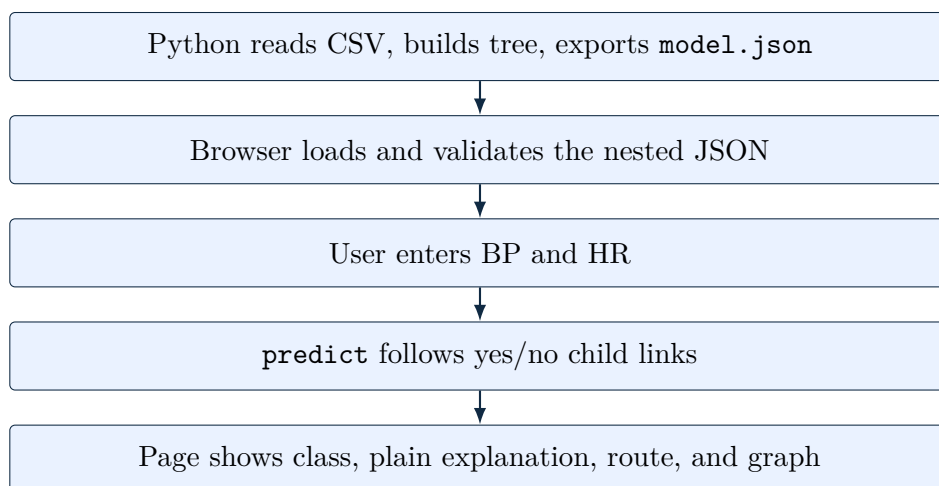


Figure 2.2: Decision-tree training and browser prediction flow.

## 2.6 Comprehensive User Manual

## 2.6.1 Recorded website experiment: the one-question blue route

Figures 2.3–2.6 deliberately separate data entry from submission. This makes it visible that changing a box is not the same action as asking the tree for a new answer.

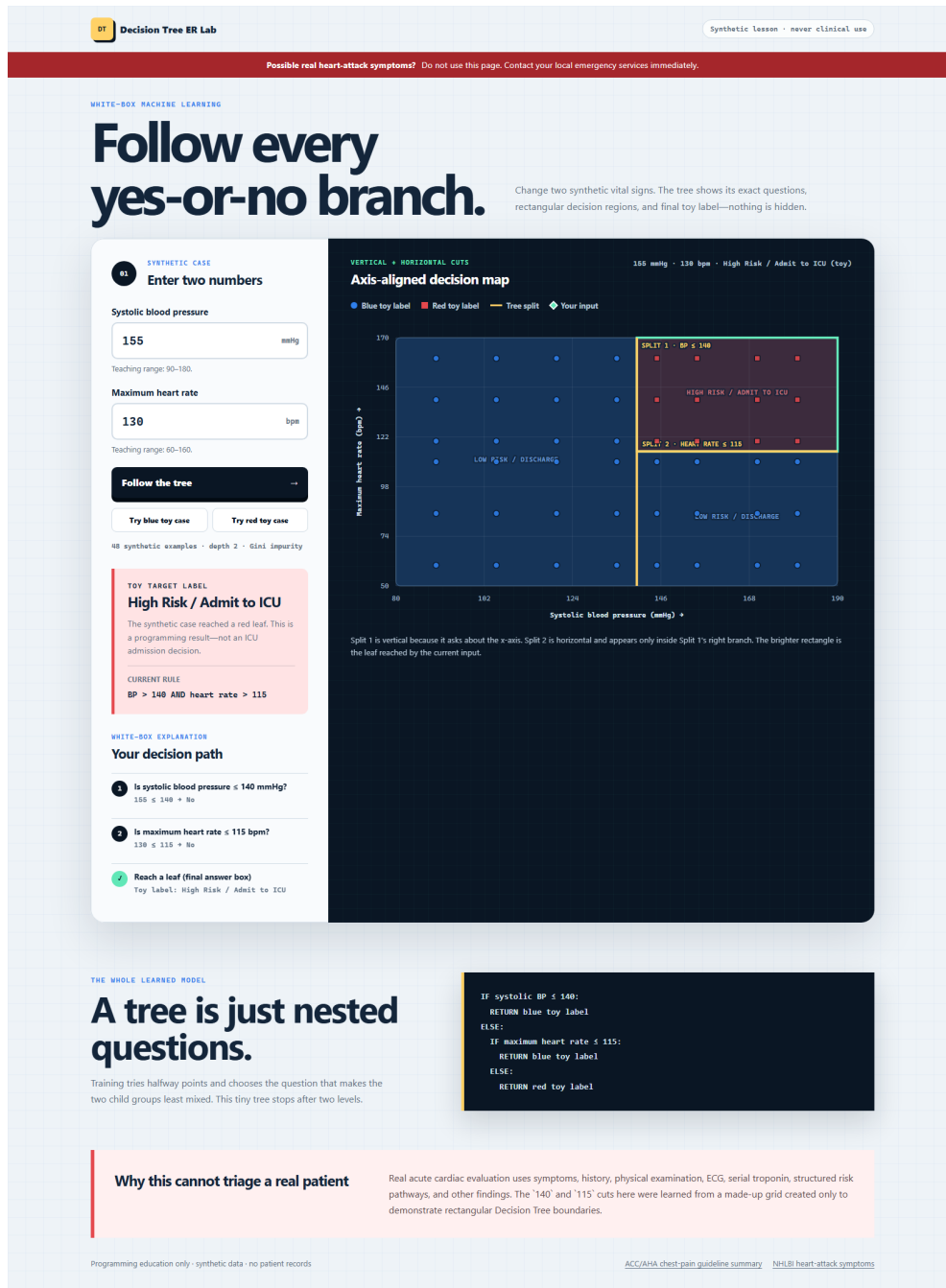


Figure 2.3: Stage 1 - initial Decision Tree page. Its default toy card is 155 mmHg and 130 bpm, so the previously rendered example occupies the red high leaf.

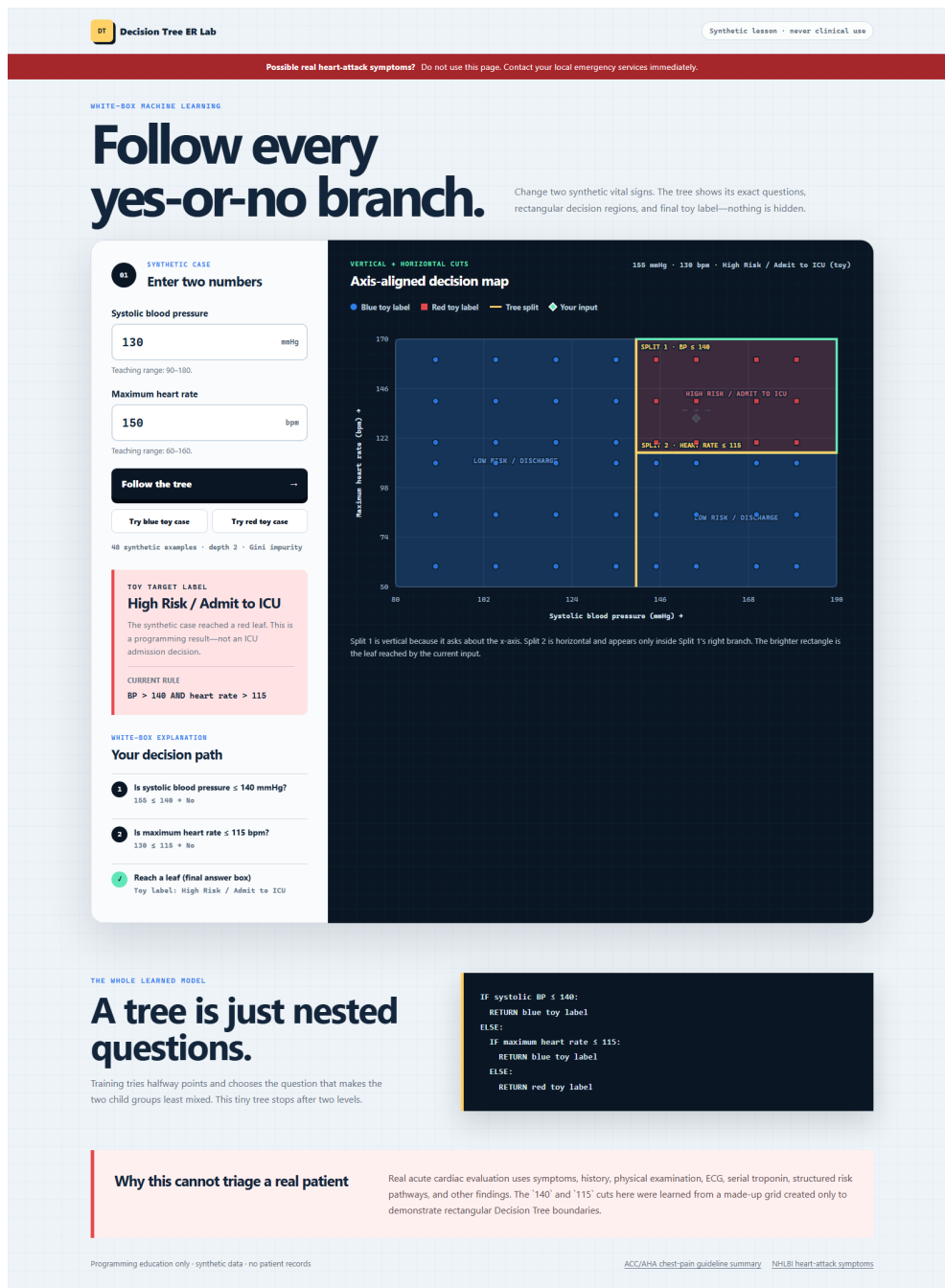


Figure 2.4: Stage 2 - new input prepared but not submitted. The boxes now contain BP 130 mmHg and maximum heart rate 150 bpm, while the earlier red result is still visible. Pressing Follow the tree is therefore a necessary function call.

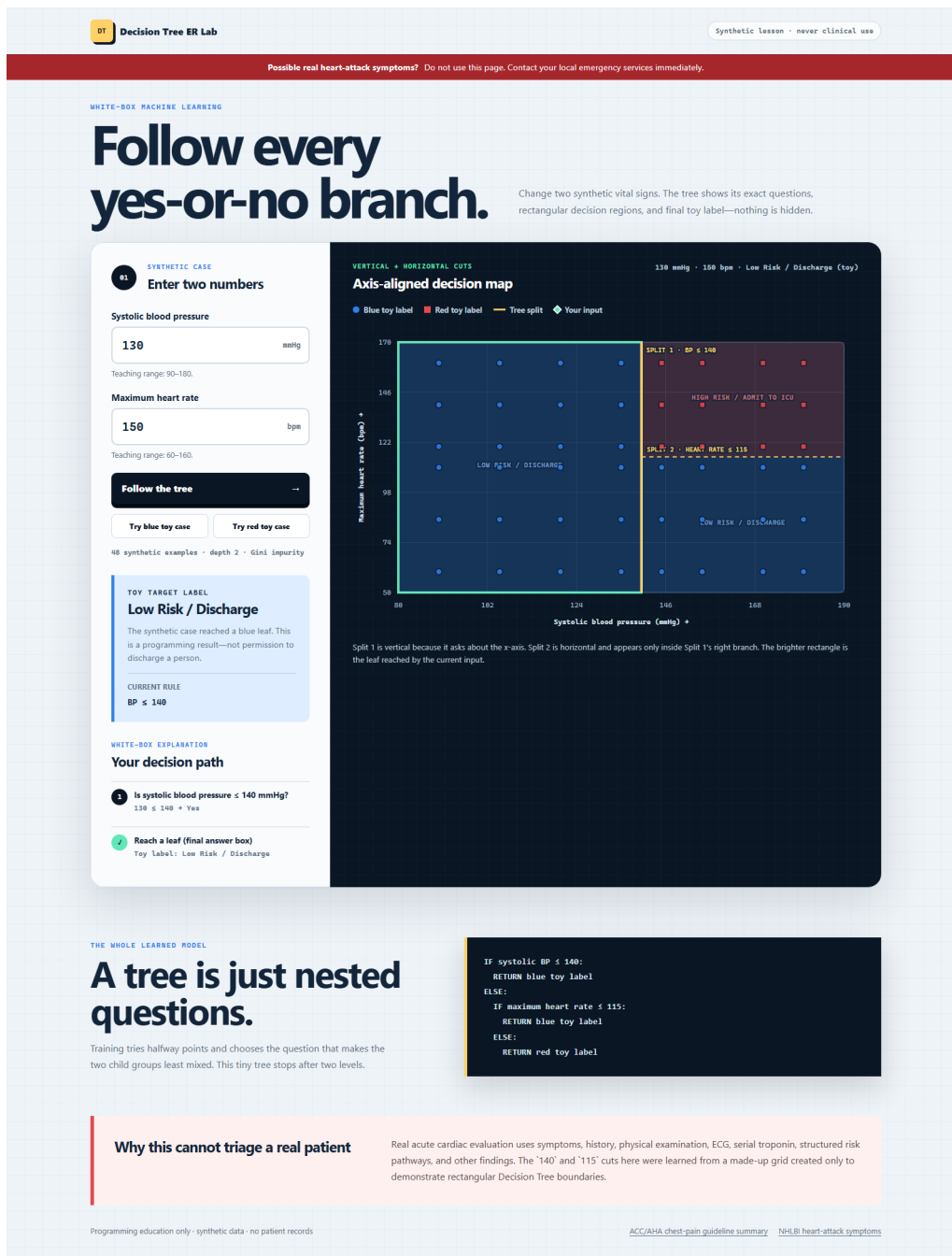


Figure 2.5: Stage 3 - Decision Tree output. For BP 130 mmHg and heart rate 150 bpm, the root question  $130 \leq 140$  is Yes, so the page returns Low Risk / Discharge as a toy label and highlights the large blue leaf.

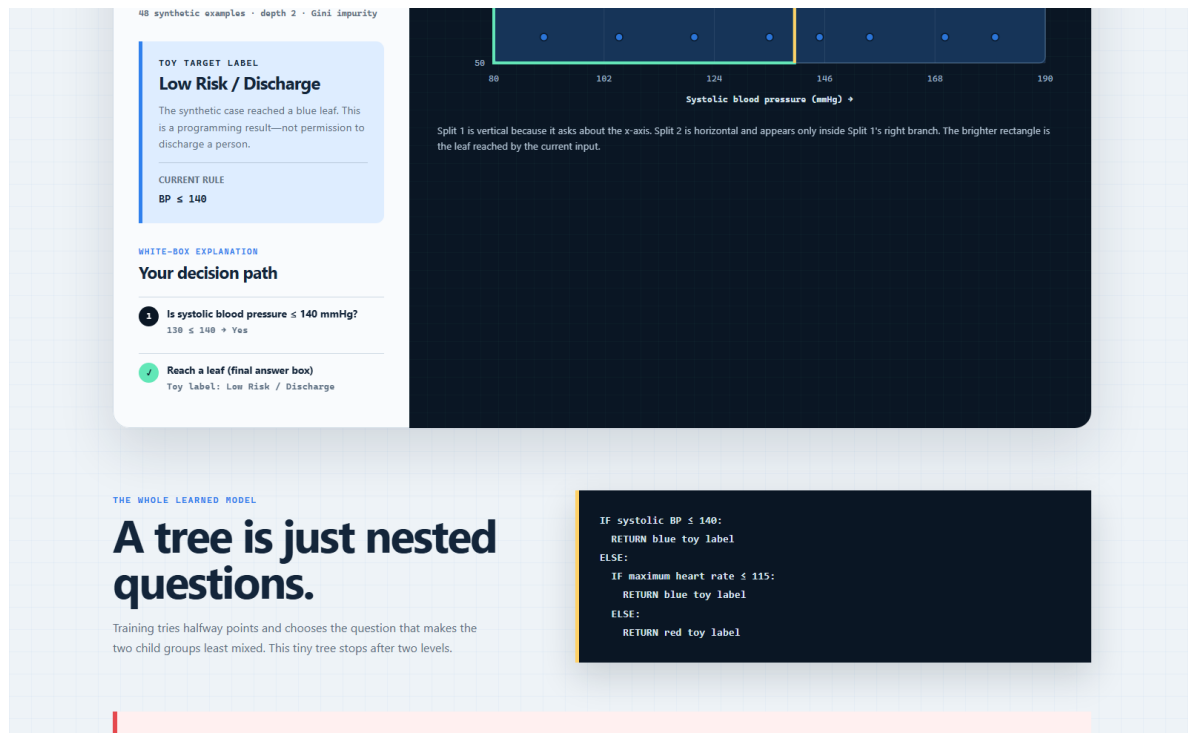


Figure 2.6: Stage 4 - focused explanation and graph. The decision path contains only the root BP comparison, and the bright outline identifies the exact rectangular leaf reached by the input point.

The recorded mapping is

$$(130 \text{ mmHg}, 150 \text{ bpm}) \longrightarrow BP \leq 140 \longrightarrow \text{Low Risk / Discharge (toy)}.$$

The heart-rate value is displayed and plotted, but the second question is not called because the first Yes branch already reaches a leaf.

## 2.6.2 Start and enter a card

Run `python start_app.py`, open the printed local address, then enter:

1. systolic BP from 90 to 180;
2. maximum HR from 60 to 160;
3. press **Follow the tree**. The preset buttons fill one low-route or one high-route example; reset restores the starting state.

## 2.6.3 Read every output

- **LOW/HIGH** is the synthetic leaf label, not a risk percentage.
- **Decision path** restates each comparison and the entered value. A yes/no marker tells which branch was taken.

- **Tree diagram** highlights visited boxes, letting a reader audit the same route visually.
- **Decision map** colors every BP–HR region and plots the current card. Its vertical and horizontal borders are learned thresholds.
- **Training accuracy** describes the 48 design cards only.

For the output in Fig. 2.5,  $130 \leq 140$ , hence the one-question LOW route. If no result appears, wait for the model status line, serve the `dist` folder instead of opening the HTML file directly, and ensure both entries are in range. If the CSV changed, regenerate JSON and refresh the page.

## 2.7 Conclusion

This lab makes model interpretability concrete: the answer is the leaf and the explanation is the route. Its very simplicity also teaches an important limit: perfect performance on designed examples is not evidence of medical validity.

## Lab 3

# Support Vector Machine

## Problem: Diabetes-Screening Margin Visualizer

### 3.1 Problem Description

This project draws a straight separating road between two synthetic groups of fasting-glucose and BMI examples. Its trained classes are **lower** and **higher**. The website also paints a narrow **follow-up** band around the road to explain uncertainty near the margin. That band is a display zone derived from score size, not a third class learned from rows.

**Safety boundary:** The output is not a diabetes diagnosis. Laboratory diagnosis uses validated tests and clinical interpretation; an abnormal result commonly requires confirmation. NIDDK lists fasting plasma glucose at 99 mg/dL or below as normal, 100–125 as prediabetes, and 126 or above as diabetes, but this synthetic two-feature SVM must not replace professional testing [6].

### 3.2 The Algorithm

#### 3.2.1 A wide road between two groups

A linear support vector machine (SVM) assigns a signed score

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{z} + b, \quad z_j = \frac{x_j - \mu_j}{s_j}, \quad (3.1)$$

where  $\mathbf{x}$  contains glucose and BMI,  $\boldsymbol{\mu}$  contains training means, and  $\mathbf{s}$  contains training standard deviations. Standardizing prevents the numerically larger glucose scale from automatically dominating. The line  $f = 0$  is the center boundary;  $f = -1$  and  $f = +1$  are margin edges. Negative scores choose **lower**; positive scores choose **higher**.

Unlike a line that merely separates, an SVM prefers a wide gap while penalizing examples that fall inside or across it. The implementation uses labels  $y \in \{-1, +1\}$  and repeatedly

reduces the regularized hinge objective

$$\lambda \|\mathbf{w}\|^2 + \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i f(\mathbf{x}_i)). \quad (3.2)$$

This is the large-margin principle introduced for support-vector networks by Cortes and Vapnik [7].

```

1 grad_w = 2.0 * regularization * weights
2 grad_b = 0.0
3 for z, y in zip(standardized_rows, labels):
4     margin = y * (dot(weights, z) + bias)
5     if margin < 1.0:                # this row still pushes on the road
6         grad_w -= (y / n) * z
7         grad_b -= y / n
8 weights -= learning_rate * grad_w
9 bias    -= learning_rate * grad_b

```

Listing 3.1: One full-batch update, simplified from `svm_model.py`.

The learning rate slowly decays during 6000 epochs so early steps travel quickly and later steps settle. The final model stores ordinary Python lists for means, scales, weights, labels, ranges, and selected display examples. A `TrainingExample` supplies `features()` and converts the word label to  $-1$  or  $+1$ .

Table 3.1: Linear-SVM functions and passed values.

| Function                     | Receives                                            | Returns or stores                         |
|------------------------------|-----------------------------------------------------|-------------------------------------------|
| <code>load_dataset</code>    | CSV path                                            | Typed list of 40 examples                 |
| <code>fit</code>             | Examples plus learning rate, regularization, epochs | Means, scales, two weights, and bias      |
| <code>decision_score</code>  | Glucose and BMI                                     | Signed distance-like score $f$            |
| <code>zone_from_score</code> | Score and display threshold                         | lower, follow-up, or higher zone          |
| <code>predict</code>         | Two raw inputs                                      | Learned binary class from the score sign  |
| <code>support_indices</code> | All trained margins                                 | Closest examples selected for the diagram |
| <code>export</code>          | Fitted model                                        | Browser-readable <code>model.json</code>  |

### 3.3 Dataset Description

The SVM data directory contains `diabetes_triage.csv`. It has 40 hand-constructed examples: 20 **lower** rows and 20 **higher** rows. Glucose spans 72–204 mg/dL and BMI spans 19–45 kg/m<sup>2</sup>. The lower group uses fasting-glucose values up to 99, while the higher group begins at 126; the omitted middle interval leaves a visible margin. The thresholds

were inspired by NIDDK’s diagnostic-test explanation [6], but the points, BMI pairings, and group labels are synthetic.

Table 3.2: Meaning of the SVM dataset columns.

| Column          | Values                  | Meaning                                                 |
|-----------------|-------------------------|---------------------------------------------------------|
| fasting_glucose | 72–204 mg/dL            | Blood-glucose concentration after fasting; feature one. |
| bmi             | 19–45 kg/m <sup>2</sup> | Mass relative to squared height; feature two.           |
| screening_label | lower/higher            | Supplied teaching class used as $-1$ or $+1$ .          |

## 3.4 Training

```

1 cd svm_diabetes_triage
2 python train_model.py
3 python start_app.py

```

The trainer standardizes both columns, starts the two weights and bias at zero, performs full-dataset gradient steps, measures training accuracy, chooses near-margin examples for display, and exports JSON. The verified current result is 100% on all 40 design rows. The exported parameters are approximately  $\mathbf{w} = (2.5947, 0.0548)$ ,  $b = 0.6838$ ,  $\boldsymbol{\mu} = (122.5875, 31.725)$ , and  $\mathbf{s} = (37.8987, 7.7168)$ .

- A larger learning rate than 0.08 moves faster but can jump back and forth; a smaller value is steadier but needs more epochs.
- More regularization than 0.02 shrinks the weights and favors a wider, simpler boundary; too much can underfit.
- More than 6000 epochs helps only until updates have settled. Always check loss or accuracy instead of assuming that more is better.
- The website’s follow-up width changes interpretation near  $f = 0$  but does not retrain the two-class SVM.

## 3.5 Website

The Python stage writes the learned numbers once. On page load, JavaScript checks that JSON contains finite two-element arrays. For each user pair it standardizes with the saved means/scales, calculates the dot product and bias, maps the score to a display zone, and moves the marker on the SVG.

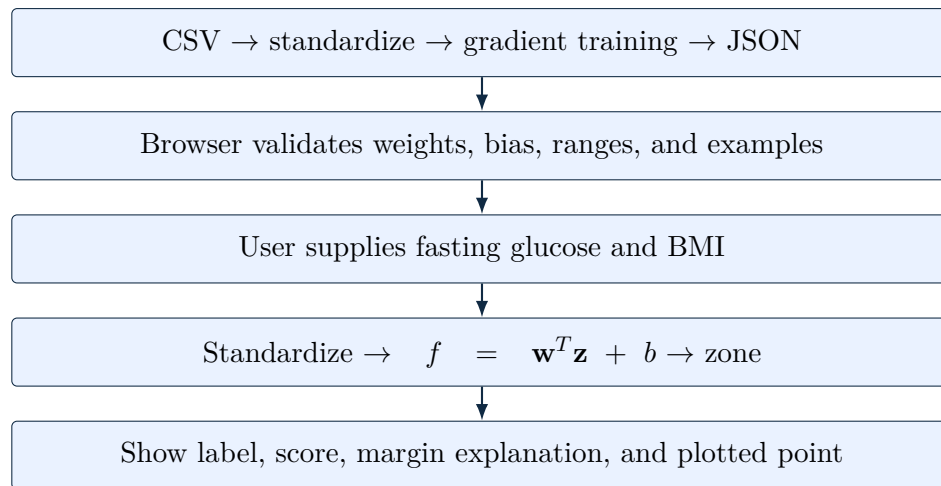
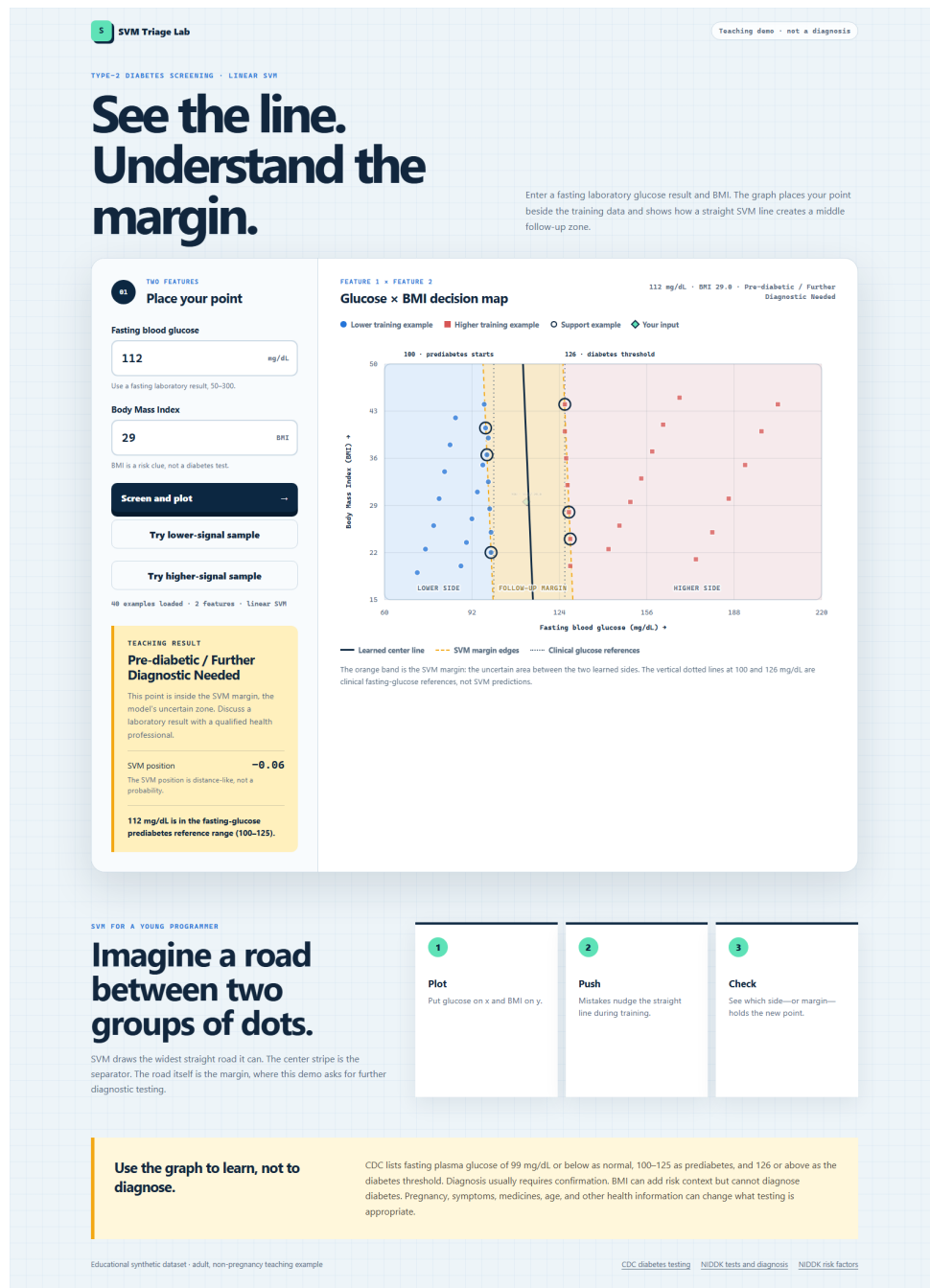


Figure 3.1: SVM training, export, and browser scoring flow.

## 3.6 Comprehensive User Manual

### 3.6.1 Recorded website experiment: a point beyond the upper margin

Figures 3.2–3.5 show the default page, the new values before submission, the returned zone, and a readable close view of the margin graph.



**SVR FOR A YOUNG PROGRAMMER**

## Imagine a road between two groups of dots.

SVR draws the widest straight road it can. The center stripe is the separator. The road itself is the margin, where this demo asks for further diagnostic testing.

**1 Plot**

Put glucose on x and BMI on y.

**2 Push**

Mistakes nudge the straight line during training.

**3 Check**

See which side—or margin—holds the new point.

**Use the graph to learn, not to diagnose.**

CDC lists fasting plasma glucose of 99 mg/dL or below as normal, 100-125 as prediabetes, and 126 or above as the diabetes threshold. Diagnosis usually requires confirmation. BMI can add risk context but cannot diagnose diabetes. Pregnancy, symptoms, medicines, age, and other health information can change what testing is appropriate.

Educational synthetic dataset - adult, non-pregnancy teaching example

[CDC diabetes testing](#) [NIDDK tests and diagnosis](#) [NIDDK risk factors](#)

Figure 3.2: Stage 1 - initial SVM page. The default example is 112 mg/dL fasting glucose and BMI 29; the full page already explains the learned line, two margins, support examples, and safety boundary.

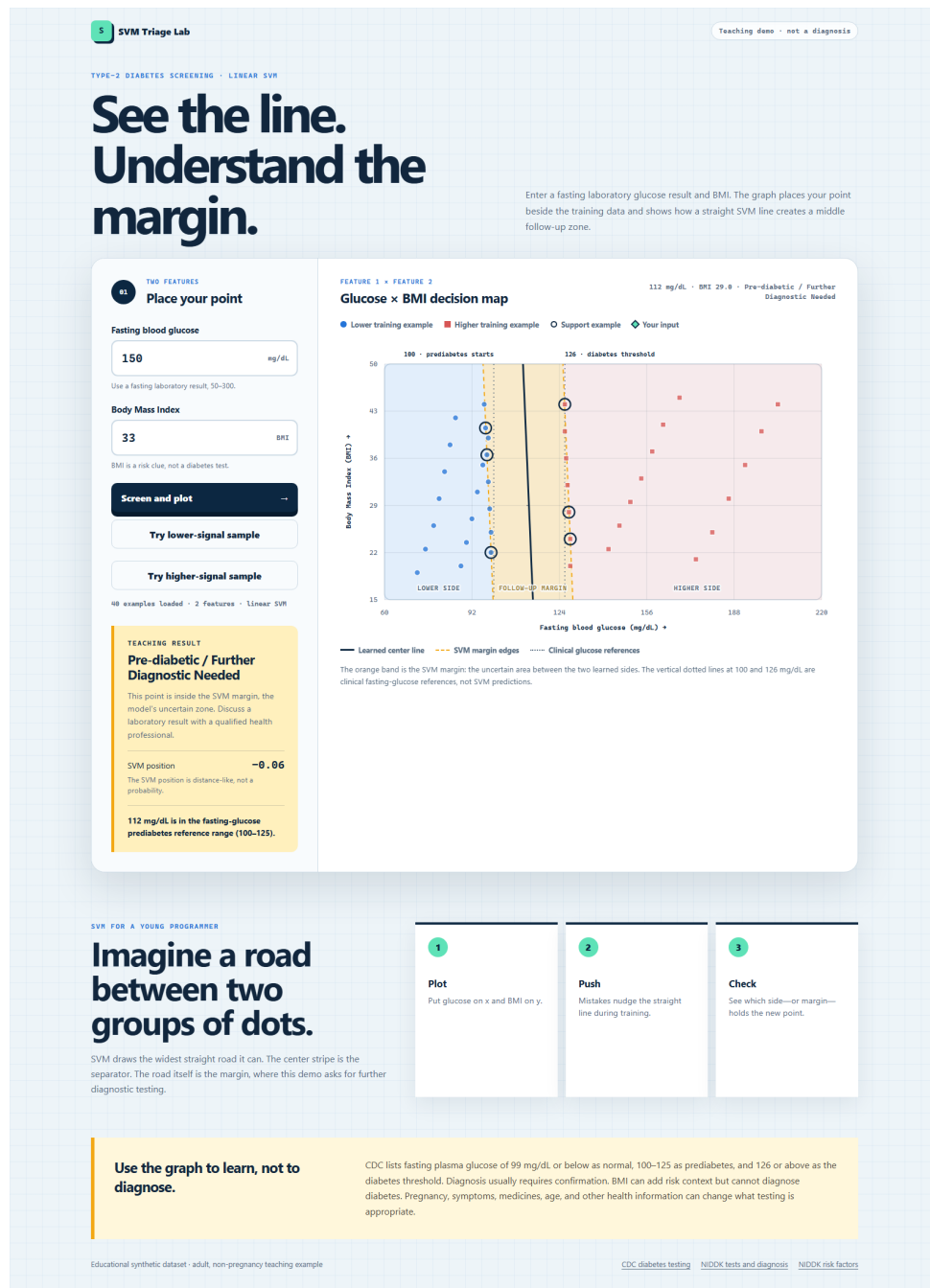


Figure 3.3: Stage 2 - input prepared. The form now contains 150 mg/dL and BMI 33, but the previous result remains until Screen and plot is pressed.

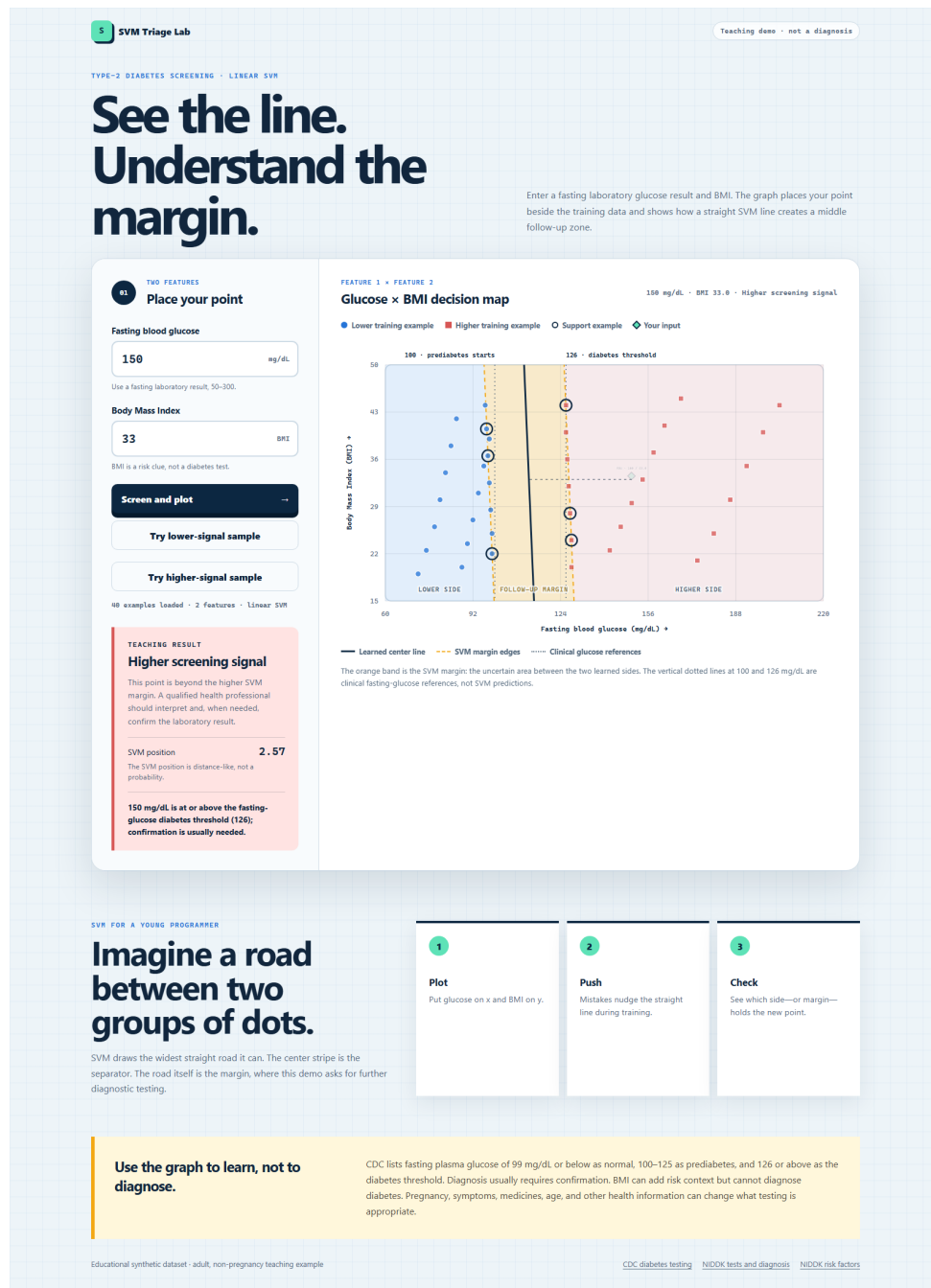


Figure 3.4: Stage 3 - SVM output for glucose 150 mg/dL and BMI 33. The signed decision score is 2.57, so the page reports Higher screening signal and moves the user marker to the red side beyond the upper margin.

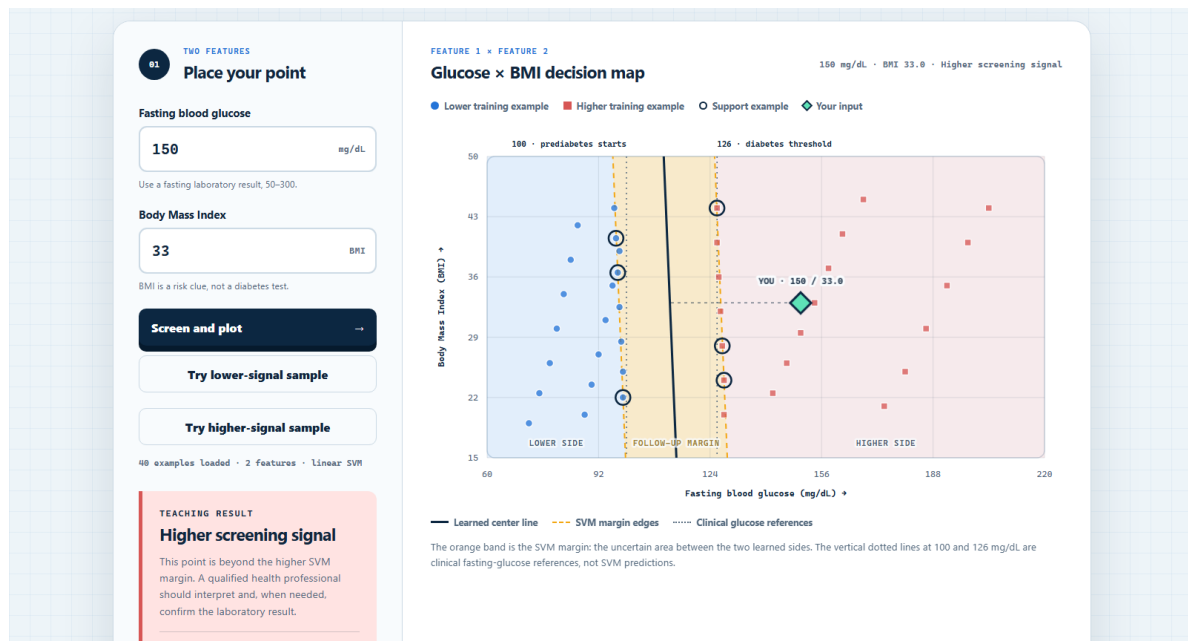


Figure 3.5: Stage 4 - focused SVM graph. The green diamond is the submitted input; blue and red points are synthetic training examples, ringed points are support examples, and the central strip shows the follow-up margin.

The recorded mapping is

$$(150 \text{ mg/dL}, 33 \text{ kg/m}^2) \longrightarrow f(\mathbf{x}) = 2.57 \longrightarrow \text{Higher screening signal.}$$

The score is a signed model value, not a probability and not a diagnosis.

Start `python start_app.py`. Enter fasting glucose from 50 to 300 mg/dL and BMI from 12 to 60 kg/m<sup>2</sup>, or select a preset, and press **Screen and plot**. Reset restores the demonstration defaults.

- **Lower-side/Higher-side** tells which side of  $f = 0$  contains the point; it does not state a probability or diagnosis.
- **Follow-up zone** means  $|f|$  is within the configured display margin. It deliberately says the example lies close to the road.
- **Decision score** is signed. Its sign chooses the side and its absolute size measures separation in the model's standardized coordinate system; it is not "percent confidence."
- **Boundary map** shows blue lower and orange higher areas, the central line, margin lines, training points, near-margin examples, and the user's larger marker.
- **Equation/explanation** substitutes the saved parameters so an advanced reader can reproduce the calculation.

In Fig. 3.4, the marker is on the higher side. If loading fails, serve the folder through the start script and regenerate `model.json`. If an input is rejected, check units and allowed range. Treat any health concern as a reason to use proper clinical care, never as a reason to tune this demo.

## 3.7 Conclusion

The SVM project links geometry, optimization, and explanation: standardized features enter one equation and its sign selects a side. The visible margin also demonstrates why near-boundary answers deserve caution.

## Lab 4

# K-Means Clustering

## Problem: Customer-Segment Explorer

### 4.1 Problem Description

A shop may have customers but no ready-made group labels. This project asks the computer to discover two patterns from each fictional customer's age and spending score. It then lets a user place a new customer on the same map and see the nearer group. The two friendly names, *Budget Explorers* and *Premium Loyalists*, are attached after clustering according to which center has the lower or higher spending score; they are interpretations, not labels shown during learning.

### 4.2 The Algorithm

#### 4.2.1 Move two team flags until nobody changes team

K-Means begins with  $K$  center points. It alternates two simple steps:

1. **Assignment:** send every point to its nearest center.
2. **Update:** move each center to the arithmetic mean of its team's coordinates.

The cycle ends when no assignment changes or a safety limit is reached. This is the iterative clustering idea described by MacQueen [8].

Age and spending use different ranges, so the implementation first compares normalized differences. For point  $x = (a, s)$  and center  $\mu = (a_c, s_c)$ ,

$$d(x, \mu) = \sqrt{\left(\frac{a - a_c}{52}\right)^2 + \left(\frac{s - s_c}{99}\right)^2}. \quad (4.1)$$

Here 52 is the page's age span (18–70) and 99 is its score span (1–100). Without normalization, a unit of age and a unit of score would be treated as equally large even though their possible ranges differ.

```

1 for (let iteration = 0; iteration < 12; iteration += 1) {
2   const next = customers.map(point =>
3     normalizedDistance(point, centers[0]) <=
4     normalizedDistance(point, centers[1]) ? 0 : 1);
5
6   centers = centers.map((oldCenter, cluster) => {
7     const members = customers.filter((_, i) => next[i] === cluster);
8     return members.length === 0 ? oldCenter : {
9       age: members.reduce((sum, p) => sum + p.age, 0) / members.length,
10      score: members.reduce((sum, p) => sum + p.score, 0) / members.length
11    };
12  });
13  if (next.every((cluster, i) => cluster === assignments[i])) break;
14  assignments = next;
15 }

```

Listing 4.1: The K-Means loop, simplified from `kmeans/dist/app.js`.

An array of 27 objects stores the customers. Another integer array stores team numbers 0 or 1, while a two-object array stores centers. These small structures make assignment by index explicit. The current initial centers are (25, 22) and (56, 76). After two rounds the final groups contain 12 and 15 points, with centers (31.67, 32.08) and (49.27, 65.93) respectively.

Table 4.1: K-Means browser functions.

| Function                        | Receives                                         | Produces or calls                                  |
|---------------------------------|--------------------------------------------------|----------------------------------------------------|
| <code>normalizedDistance</code> | A point and a center                             | One Euclidean distance after scaling               |
| <code>runKMeans</code>          | Customer array, starting centers, maximum rounds | Final centers, assignments, and replay snapshots   |
| <code>getTeamRoles</code>       | Final centers                                    | Which team gets the budget/premium display name    |
| <code>classifyCustomer</code>   | User age and score                               | Nearest center, segment, distance, and explanation |
| <code>setInputs</code>          | A preset point                                   | Updates both sliders/numbers and validation        |
| <code>startReplay</code>        | Saved snapshots and speed                        | Animates assignment and center movement            |
| <code>draw helpers</code>       | Model and selected point                         | Customer map, boundaries, centers, and legend      |

## 4.3 Dataset Description

There is no CSV and no imported customer information. The 27 fictional records are declared directly in `kmeans/dist/app.js`. They were arranged to make two partly separated teaching groups across the accepted ranges. Because K-Means is unsupervised, no answer column is provided.

Table 4.2: Fields in each fictional customer object.

| Field    | Range       | Plain-language meaning                                                                                        |
|----------|-------------|---------------------------------------------------------------------------------------------------------------|
| age      | 18–70 years | Horizontal position of a fictional adult customer.                                                            |
| score    | 1–100       | Artificial shopping-engagement/spending score; a larger number means stronger activity in this demonstration. |
| No label | —           | The missing answer is the point: K-Means must form teams itself.                                              |

To recreate or expand the data, add objects such as `{age: 35, score: 44}` to the `customers` array, keep values in range, reload, and inspect whether the final grouping is stable. Real customer segmentation would require lawful, representative, privacy-conscious data and a business definition of a useful segment.

## 4.4 Training

Training occurs in JavaScript as soon as the page loads; no Python model file exists. From the project folder:

```

1 cd kmeans
2 npm start
3 # Open the printed localhost address; loading the page runs K-Means.
```

The current run stabilizes after two rounds. To retrain, change the customer array, starting centers, or algorithm settings and refresh.

- $K$  is fixed at 2. Supporting  $K = 3$  requires a third center, a general nearest-center loop, three colors/names, and UI changes.
- Different starting centers can lead to a different local solution. Testing several starts and keeping the smallest within-cluster squared error is a common improvement.
- The maximum of 12 rounds is only a guard. Raising it matters if a modified dataset has not stabilized; it does not force extra movement after convergence.
- Removing normalization gives the feature with the larger numerical variation more influence. Changing the age/score spans changes the meaning of distance.

## 4.5 Website

The entire pipeline is local to the page. The saved snapshots make an animation possible without rerunning a different model during replay.

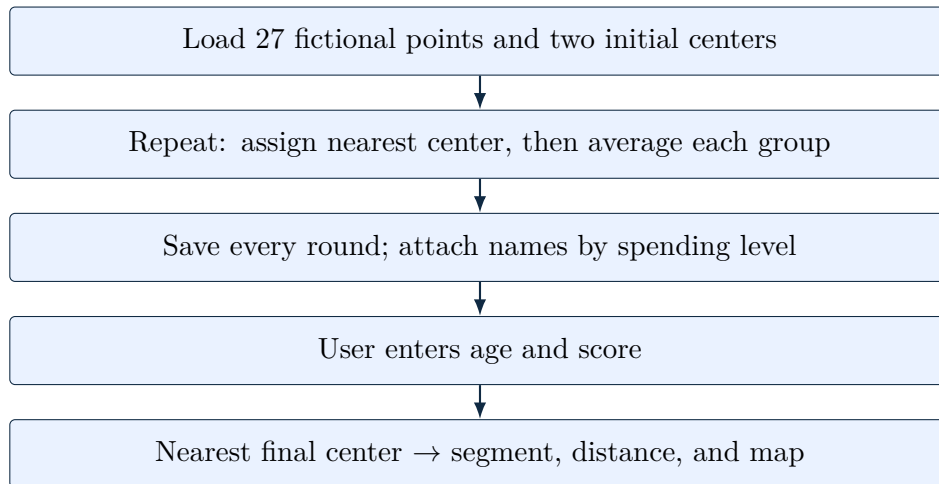


Figure 4.1: K-Means page-load training and user classification flow.

## 4.6 Comprehensive User Manual

### 4.6.1 Recorded website experiment: assign a new customer and replay learning

Figures 4.2–4.5 record both the final classification and an earlier training frame, so prediction and cluster learning are not confused.

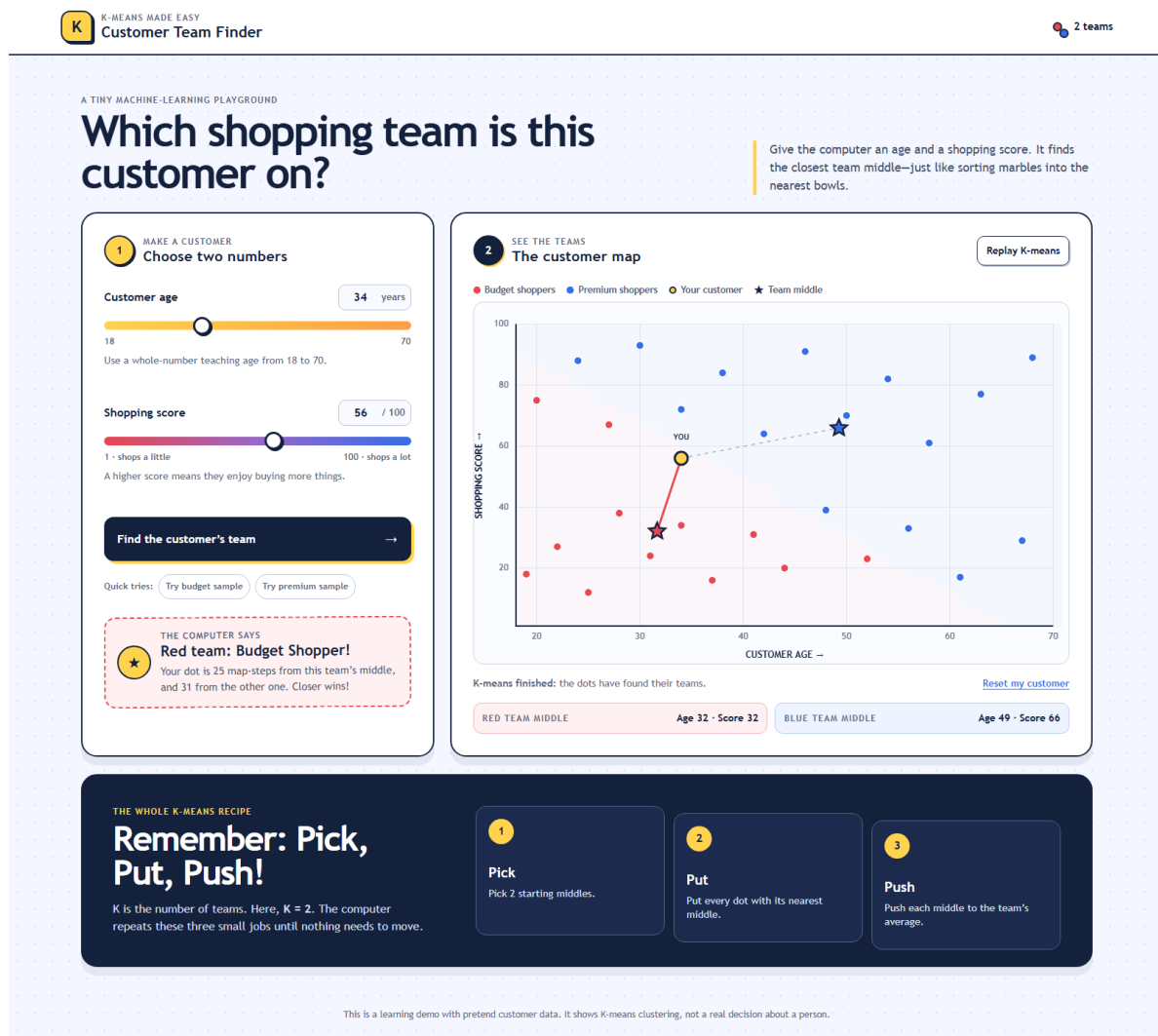


Figure 4.2: Stage 1 - initial K-Means page. The starting demonstration uses age 34 and shopping score 56 and shows the trained red/blue groups, their centroid stars, and a ready-made red-team result.

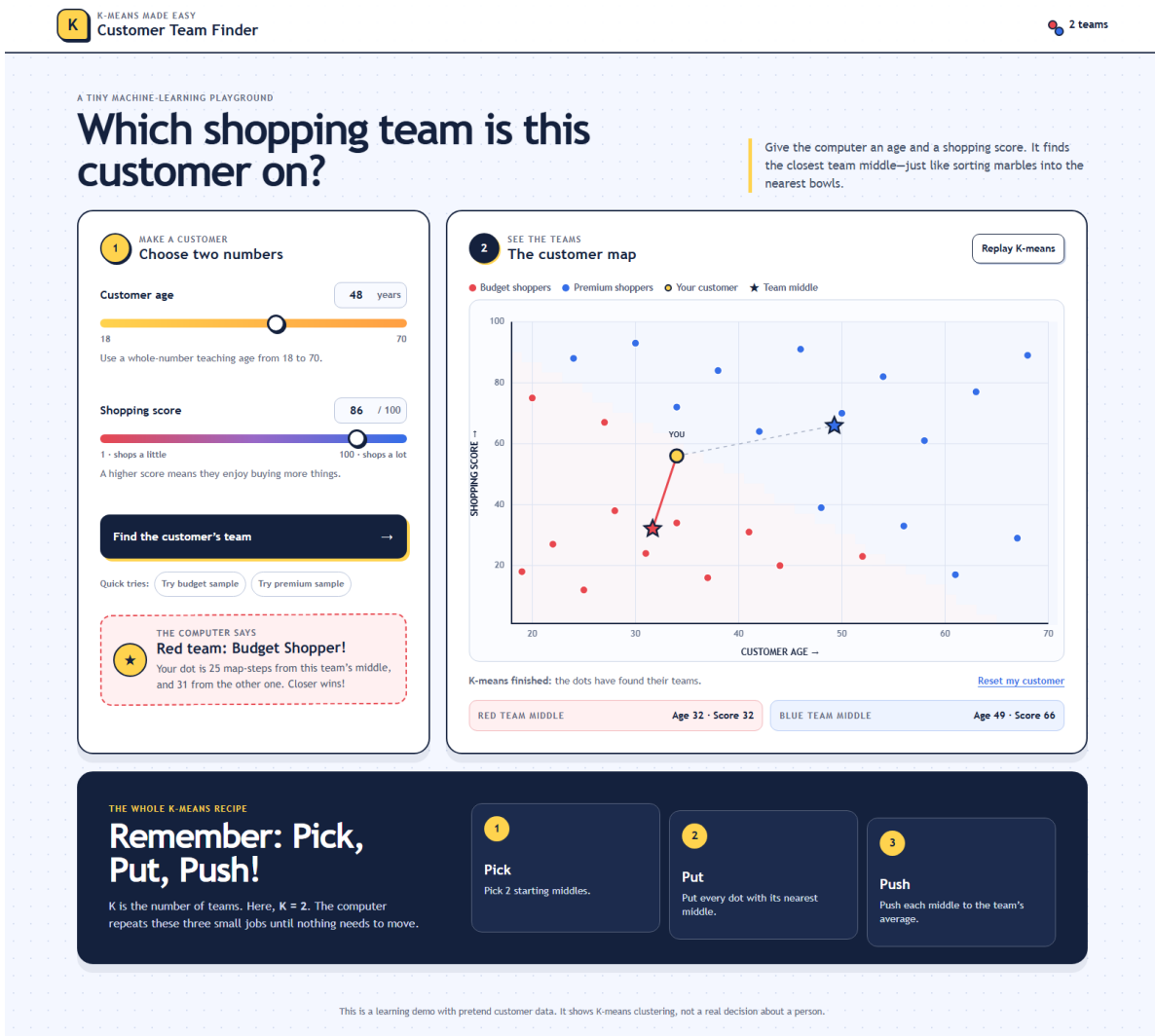


Figure 4.3: Stage 2 - input prepared. The controls show age 48 and shopping score 86, while the old red-team answer is still visible because the form has not yet been submitted.

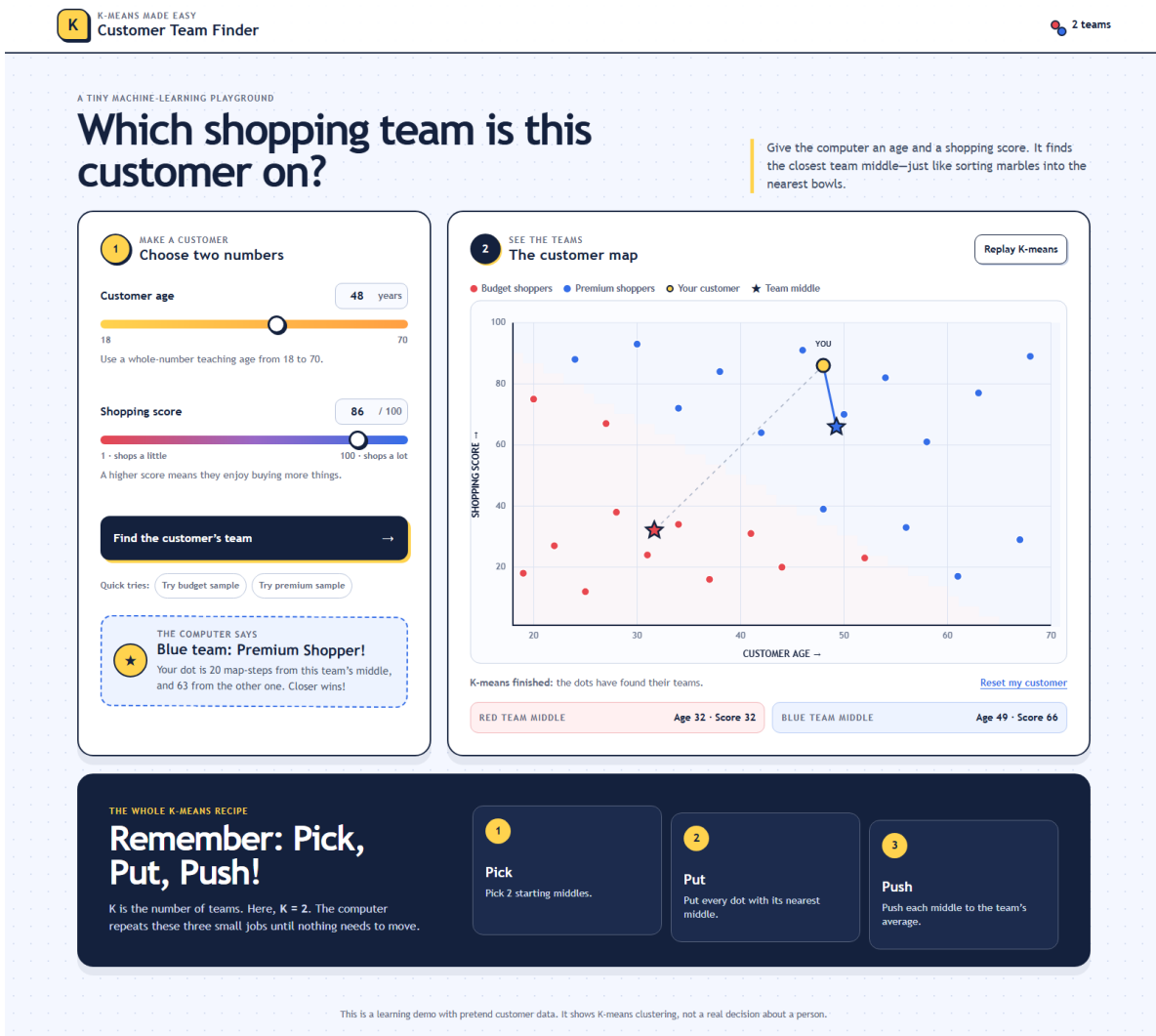


Figure 4.4: Stage 3 - K-Means output. Age 48 and score 86 are assigned to Blue team: Premium Shopper. The page reports a normalized map distance of 20 to the blue center and 63 to the red center; the smaller distance wins.

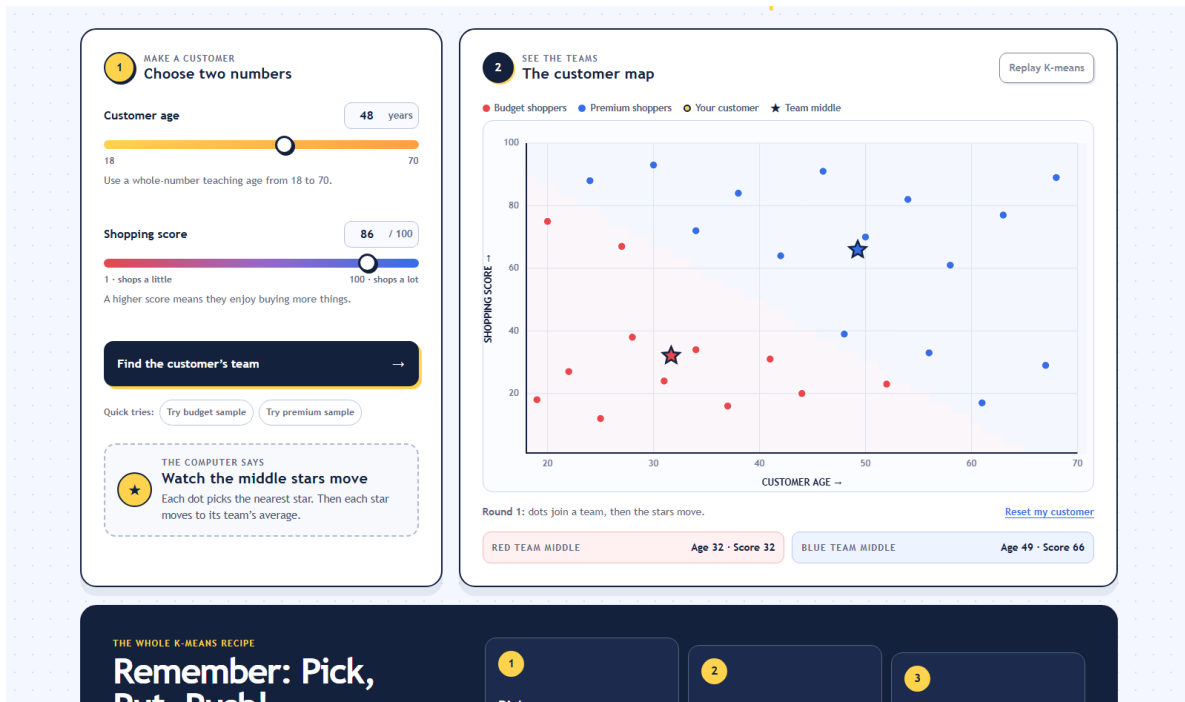


Figure 4.5: Stage 4 - additional training information. During replay round 1, every saved dot joins its nearer star and the centroids move toward their current team averages. This animation changes the displayed learning frame, not the final answer.

The displayed final centers are approximately (32, 32) for the red group and (49, 66) for the blue group. Thus the visible record is  $(48, 86) \rightarrow d_{blue} = 20 < d_{red} = 63 \rightarrow \text{Blue team}$ .

1. Start with `npm start` and open the page.
  2. Enter an age from 18 to 70 and score from 1 to 100, or use a preset. Inputs are linked to sliders, so moving either representation updates the other.
  3. Press **Classify customer**. Use **Replay K-Means** to watch center movement; replay changes the visualization, not the final model.
  4. Press reset to restore the original example and camera state.
- **Segment name** is a friendly description assigned after clustering. It is not a moral judgment and was not present in training.
  - **Cluster number/color** is the algorithm's team identity. Swapping numerical identities changes no grouping meaning.
  - **Centroid** is the average age and score of all current members. It may not be an actual customer.
  - **Distance** is the normalized separation between the new point and its selected centroid; smaller means more similar under these two fields. It is not probability.
  - **Map** plots data dots, centroid stars, the decision boundary, and the entered point. Hover details identify values and team.
  - **Iteration cards** explain which round changed assignments.

Figure 4.4 places age 48, score 86 with the higher-spending center. If nothing draws, use a current browser with JavaScript enabled and serve the page rather than relying on restrictive local-file access. If a number is red, bring it inside the printed range.

## 4.7 Conclusion

This explorer clearly separates discovery from naming: K-Means finds compact groups using distance, then people decide whether those groups are meaningful. The replay turns an abstract iterative calculation into visible movement.

## Lab 5

---

# CNN and YOLO Object Detection

---

## Problem: Sightline Object Detector

### 5.1 Problem Description

People can glance at a photograph and say both *what* they see and *where* it is. A computer receives only a rectangular grid of red, green, and blue numbers. This application accepts one photograph or camera frame, finds recognizable objects, draws a box around each one, names it, and reports a confidence score. Unlike ordinary image classification, object detection may return zero, one, or many located answers.

### 5.2 The Algorithm

#### 5.2.1 From pixels to visual clues

A convolutional neural network (CNN) slides small learned filters over an image. For output position  $(i, j)$  and filter  $k$ , a simplified operation is

$$y_{i,j,k} = \sigma \left( b_k + \sum_{u,v,c} W_{u,v,c,k} x_{i+u,j+v,c} \right). \quad (5.1)$$

The input  $x$  is the pixel array,  $W$  and  $b$  are learned filter values, and  $\sigma$  is a nonlinear activation. Early layers react to edges and colors; deeper combinations can represent textures, parts, and object shapes. Training has already adjusted millions of such numbers in the supplied weight file.

#### 5.2.2 YOLO detection in one model pass

YOLO means “You Only Look Once.” Its central idea is to predict locations and classes from the whole image in a unified pass rather than running a classifier separately over

many proposed crops [9]. The current project uses the pretrained Ultralytics YOLOv8 nano checkpoint `yolov8n.pt`, a compact member of the YOLOv8 detection family [10].

For each candidate, the network produces four box coordinates, an object class, and a confidence. Low-confidence candidates are discarded. Overlapping guesses for the same object are reduced by non-maximum suppression inside the library; the retained guesses become `result.bboxes`. The application converts each tensor-like box into a plain dictionary:

```

1 results = model.predict(
2     source=image,           # Pillow RGB image
3     conf=confidence,        # accepted range: 0.10 to 0.90
4     imgsz=int(os.getenv("YOLO_IMAGE_SIZE", "640")),
5     verbose=False,
6 )
7 for box in results[0].boxes:
8     class_id = int(box.cls[0].item())
9     score = float(box.conf[0].item())
10    x1, y1, x2, y2 = box.xyxy[0].tolist()
11    detections.append({
12        "label": results[0].names[class_id],
13        "confidence": round(score, 4),
14        "box": [round(x1), round(y1), round(x2), round(y2)]
15    })

```

Listing 5.1: Inference and result conversion from `cnn_object_detector/detector.py`.

**Plain meaning.** A returned record such as `dog, 0.91, [20,40,500,690]` says: the model guessed “dog” with 91% model confidence, and the rectangle runs from pixel (20,40) to pixel (500,690). Confidence is not a promise that the label is correct.

The `YoloDetector` loads the heavy model only on first use and keeps it for later requests. A re-entrant lock prevents two requests from using the shared model simultaneously. `DetectionResult` is an immutable data class holding annotated JPEG bytes and a list of dictionaries. Detections are sorted strongest-first, assigned visible colors, and passed to `_annotate`, which draws matching boxes and labels on a copy.

Table 5.1: CNN web application functions and information flow.

| Function                       | Receives                             | Returns or calls                                                        |
|--------------------------------|--------------------------------------|-------------------------------------------------------------------------|
| <code>_read_image</code>       | Uploaded byte stream                 | Validated, oriented, bounded RGB Pillow image                           |
| <code>_parse_confidence</code> | Form text                            | Float from 0.10 through 0.90                                            |
| <code>_get_model</code>        | Model filename/environment           | Lazily loaded shared YOLO object                                        |
| <code>detect</code> (detector) | RGB image and confidence             | Calls <code>model.predict</code> ; returns <code>DetectionResult</code> |
| <code>_annotate</code>         | Image copy and detection list        | JPEG-ready image with colored boxes                                     |
| <code>/api/detect</code>       | Multipart image and confidence       | JSON containing data-URL image, boxes, count, time, size, and model     |
| <code>run</code> (browser)     | Selected file/camera blob and slider | Calls API, then renders image, cards, and statistics                    |

## 5.3 Dataset Description

### 5.3.1 What data the current application uses

The website does not contain a local training CSV and does not add uploaded photos to a dataset. Its learned values came with a model pretrained on the COCO object-detection benchmark. COCO contains complex everyday scenes with large-scale instance annotations [11]; the Ultralytics detection checkpoint exposes the standard 80-category detection label set [10]. Examples include person, bicycle, car, dog, chair, and bottle. An object outside those learned categories can be missed or confused.

The only current per-request “record” is:

Table 5.2: Input image information and its meaning.

| Item               | Constraint                                 | Purpose                                                |
|--------------------|--------------------------------------------|--------------------------------------------------------|
| Image bytes        | JPG, PNG, WEBP, or BMP; at most 12 MB      | The scene to analyze.                                  |
| Source dimensions  | Under 40 megapixels                        | Blocks decompression-sized images before full loading. |
| Working dimensions | Longest side at most 1920 before inference | Bounds memory while preserving aspect ratio.           |
| RGB pixels         | Three 0–255 color channels                 | Predictable image representation passed to YOLO.       |
| Confidence         | 0.10–0.90; default 0.35                    | Smallest model score allowed into the result.          |

Phone orientation is corrected from EXIF metadata, then images are converted to RGB using Pillow. Flask’s documented upload pattern motivates receiving the file through multipart form data and applying explicit size/type checks [12]; Pillow performs decoding and orientation [13].

## 5.4 Training and Model Use

### 5.4.1 Run the supplied pretrained detector

The normal laboratory workflow is inference, not training:

```

1 cd cnn_object_detector
2 python -m venv .venv
3 .venv\Scripts\Activate.ps1
4 pip install -r requirements.txt
5 python app.py
6 # Open http://127.0.0.1:5000

```

On the first detection, `_get_model` imports Ultralytics and opens the bundled `yolov8n.pt`. The API validates the upload, calls the detector, times it, Base64-encodes the annotated JPEG, and returns one JSON response. The photo is held in memory by this implementation rather than deliberately saved as a server file.

### 5.4.2 Optional custom training

If a future student creates an annotated YOLO dataset, training is a separate official Ultralytics workflow, for example [14]:

```

1 yolo detect train data=path\to\data.yaml model=yolov8n.pt epochs=50 imgsz=640

```

The YAML identifies training/validation image folders and class names; label text files supply normalized boxes and class IDs. This command is illustrative and does not describe a dataset currently present in the workspace.

- More `epochs` provides more passes over the custom training set, but excessive passes can overfit; validation metrics must decide when to stop.
- Larger `imgsz` can preserve small-object detail but costs more memory and time. The site's inference default is 640.
- A larger batch can speed stable hardware training but needs more memory. Learning-rate changes affect step size and must be tuned carefully.
- On the website, raising confidence (for example 0.35 to 0.60) usually shows fewer, stronger guesses. Lowering it shows more possibilities and more false positives. This filters inference; it does not retrain weights.

## 5.5 Website

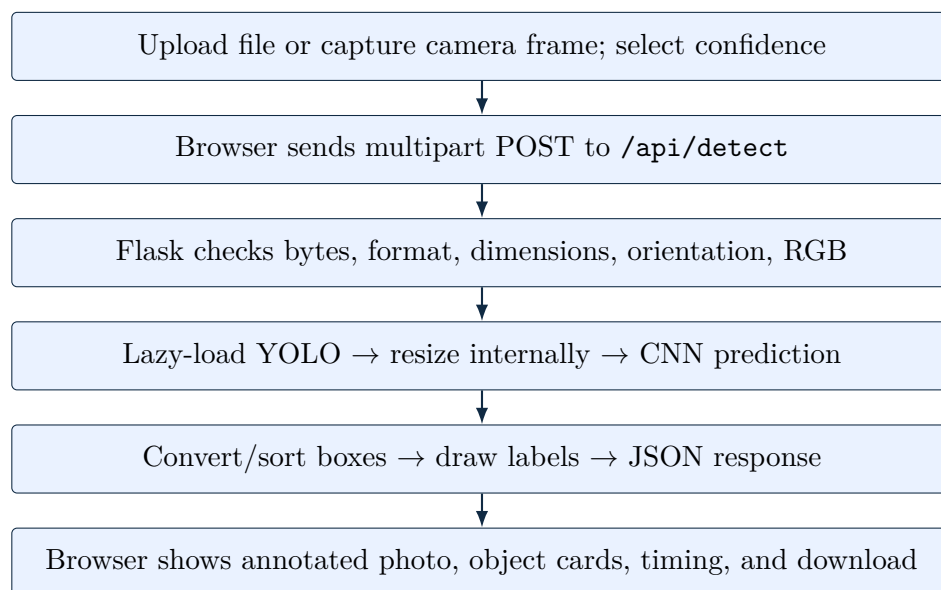


Figure 5.1: End-to-end Sightline request flow.

## 5.6 Comprehensive User Manual

### 5.6.1 Recorded website experiment: a busy urban intersection

The test photograph was made specifically for this report and contains many ordinary, visually separated objects. Figures 5.2– 5.6 show the exact browser sequence and the image before and after real YOLOv8n inference.

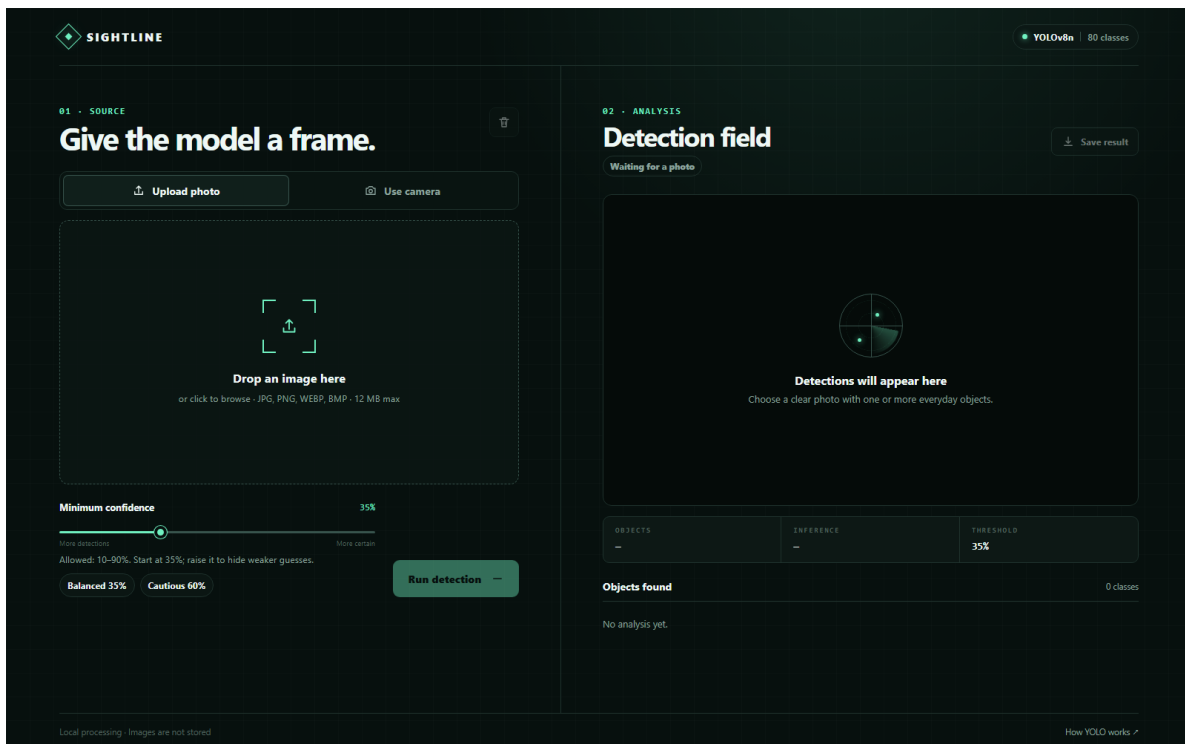


Figure 5.2: Stage 1 - initial Sightline page. No photograph is selected, the result field says Waiting for a photo, the count and time are blank, and the threshold is 35%.

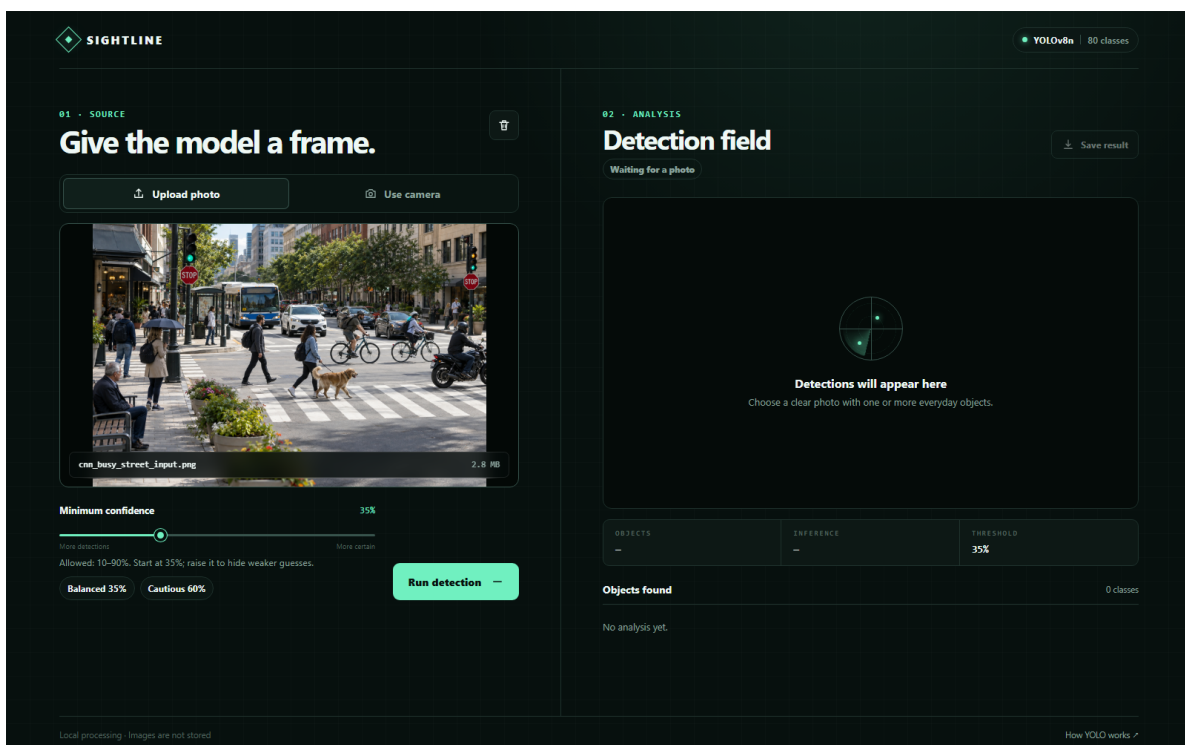


Figure 5.3: Stage 2 - input prepared. The busy street photograph appears in the upload preview with its filename and size. Run detection is enabled, but the output panel is still empty.

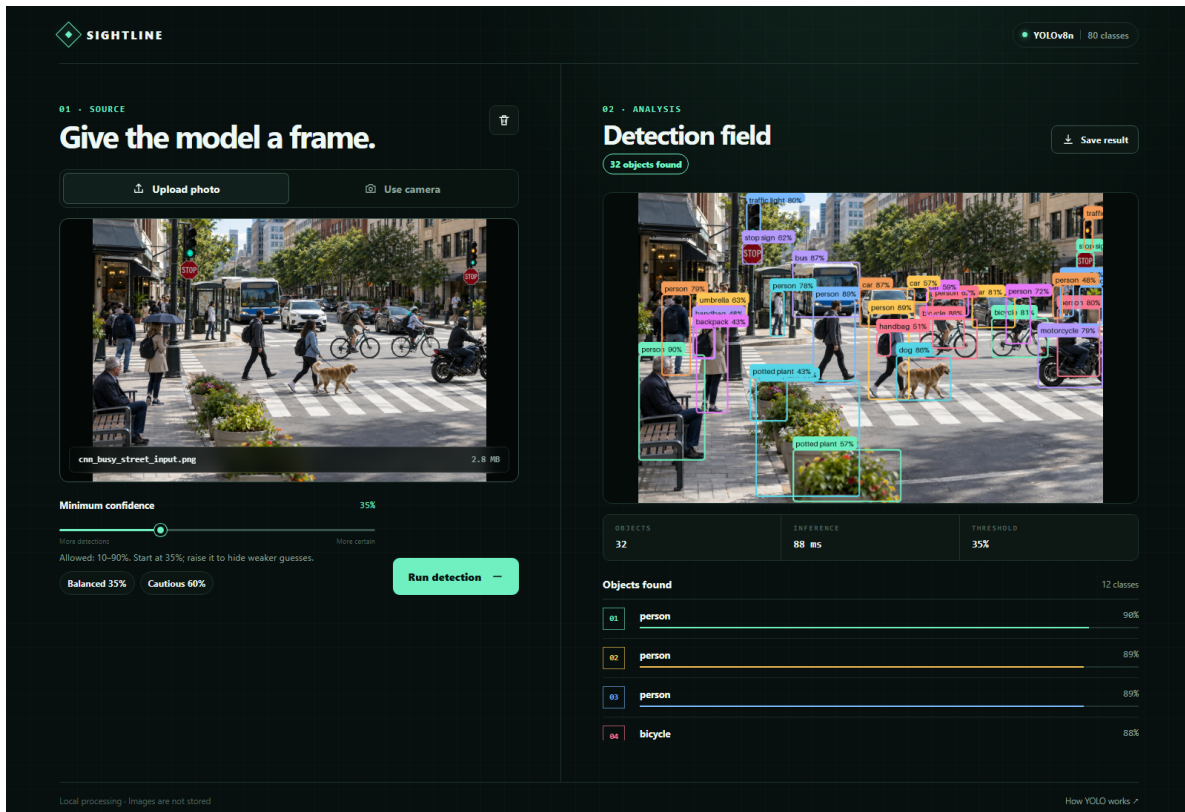


Figure 5.4: Stage 3 - live YOLO output at the 35% threshold. The annotated preview shows colored boxes and labels; the website reports 32 objects across 12 classes and an 88 ms warm-run inference time on this machine.

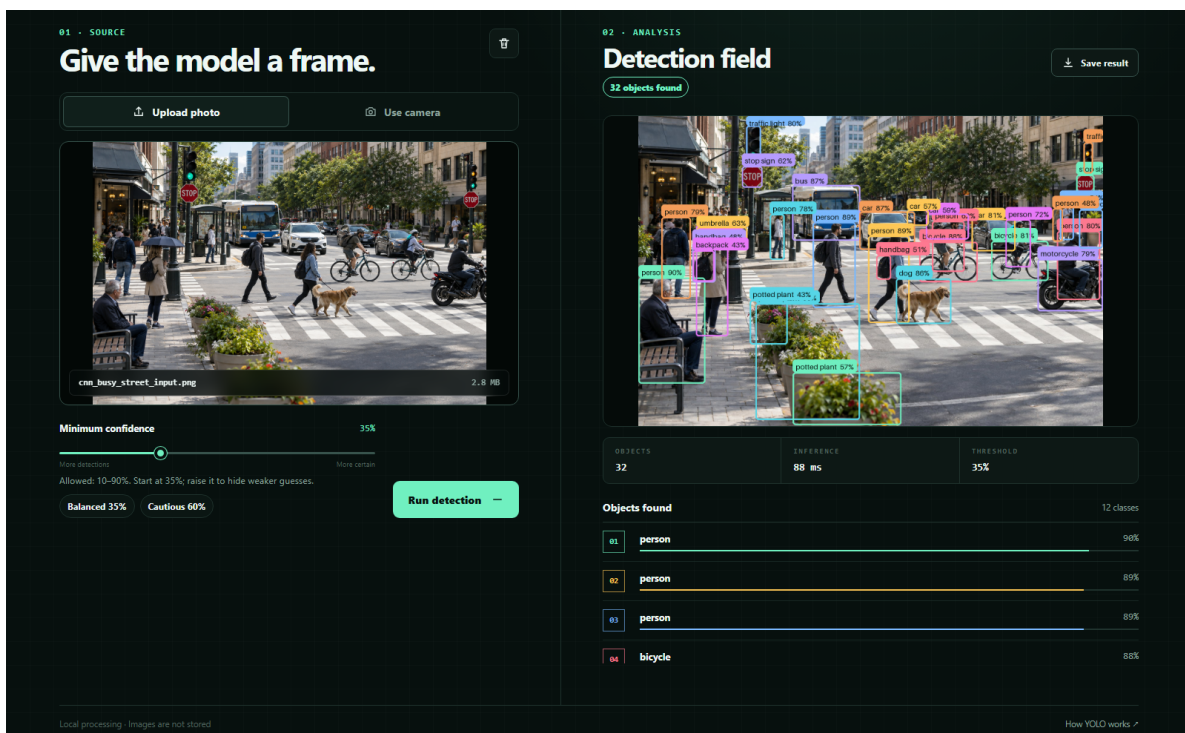
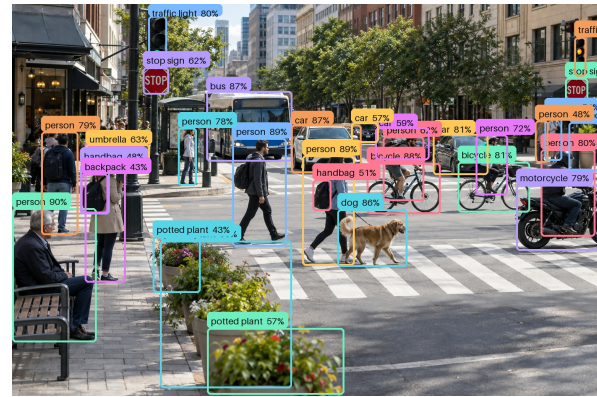


Figure 5.5: Stage 4 - focused result evidence. The result badge, annotated frame, count, time, threshold, class count, and strongest-first confidence list are visible together.



**Input:** unannotated 1536-by-1024 street scene



**Output:** YOLO boxes, labels, and confidence percentages

Figure 5.6: Exact CNN experiment before and after website inference. The input bytes on the left produced the annotated JPEG returned by the Flask API on the right.

Table 5.3: Observed output for the busy-street CNN experiment.

| Displayed output    | Recorded value and meaning                                                                                               |
|---------------------|--------------------------------------------------------------------------------------------------------------------------|
| Accepted input      | PNG street scene, 1536 by 1024 pixels, processed at a 35% minimum-confidence threshold.                                  |
| Detected boxes      | 32 boxes remained after confidence filtering and overlap suppression.                                                    |
| Unique classes      | 12: person, bicycle, bus, car, dog, traffic light, motorcycle, stop sign, umbrella, potted plant, handbag, and backpack. |
| Strongest examples  | Person 90%, person 89%, bicycle 88%, bus 87%, car 87%, and dog 86%.                                                      |
| Warm inference time | 88 ms on the capture machine; timing depends on hardware and model warm-up.                                              |

## 5.6.2 Choose an input

After starting `python app.py`, open the page and choose one of two tabs:

- **Upload photo:** click or drag a JPG, PNG, WEBP, or BMP no larger than 12 MB. A preview and filename confirm the selection.
- **Use camera:** allow browser camera permission, aim at the scene, and capture a frame. Camera access normally requires localhost or HTTPS.

Do not upload private images to a server you do not trust. This local code does not intentionally save them, but deployment operators remain responsible for transport security, logs, and retention policy.

### 5.6.3 Run and understand the answer

Set the confidence slider from 10% to 90% (35% is a useful starting point), then press **Detect objects**.

- **Annotated photo:** each colored rectangle is a predicted object location. Its tag contains class name and rounded confidence.
- **Detected objects:** a strongest-first list. Matching colors link list cards to boxes.
- **Count:** number of boxes surviving the chosen confidence and the model's suppression process, not necessarily the true object count.
- **Time:** server inference elapsed milliseconds. It excludes some browser/network work and varies by CPU/GPU and first model load.
- **Image size/model/threshold:** records the exact request context.
- **Save result:** downloads the annotated JPEG; reset clears the current input and output.

If the output says no objects, try a clear, well-lit photo, reduce confidence a little, or remember that the object may not be one of the 80 known categories. If the model cannot start, activate the environment, install requirements, and confirm `yolov8n.pt` is readable. A 400 error indicates an invalid input; 413 means too large; 503 means the detector could not load/analyze. The first successful run is normally slower because the model changes from “cold” to “loaded.”

## 5.7 Conclusion

Sightline joins a pretrained visual learner to a defensive, understandable web pipeline. The lab distinguishes pixels, learned CNN features, YOLO detections, application-side validation, and user-facing evidence while making the limits of confidence and label coverage explicit.

---

## Overall Conclusion

---

The five laboratory problems cover complementary learning strategies. KNN consults nearby stored examples; the Decision Tree follows auditable questions; the SVM learns a wide linear margin; K-Means discovers unlabeled groups; and CNN/YOLO turns pixels into multiple located object hypotheses. Their websites expose the evidence most natural to each model: neighbors, a branch path, a margin, moving centroids, or bounding boxes.

Across all five projects, a model's output inherits the limits of its data and objective. Synthetic labels can teach mechanics but cannot establish clinical truth. Training-set accuracy cannot replace independent evaluation, an SVM score is not automatically a probability, and a cluster is not automatically meaningful. Keeping those distinctions beside the interactive controls is as important as implementing the formula.

## Appendix A

# Verification and Reproduction

## A.1 Commands verified in this workspace

The following checks were executed against the submitted sources before this report was produced.

| Project        | Verification command                                                                                                                           | Observed result                      |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| KNN            | <code>python -m unittest discover -s tests -p test_knn.py -v;</code><br><code>tests/test_browser_model.js</code><br><code>node</code>          | 4 Python tests; browser tests passed |
| Decision Tree  | <code>python -m unittest discover -s tests -p test_decision_tree.py -v;</code><br><code>tests/test_browser_tree.js</code><br><code>node</code> | 5 Python tests; browser tests passed |
| SVM            | <code>python -m unittest discover -s tests -p test_svm.py -v;</code><br><code>tests/test_browser_model.js</code><br><code>node</code>          | 4 Python tests; browser tests passed |
| K-Means        | <code>node -check dist/app.js</code>                                                                                                           | Syntax check passed                  |
| CNN/YOLO layer | <code>python -m pytest tests/test_app.py -q</code>                                                                                             | 4 tests passed                       |

The KNN retraining command reported 60 rows, a 48/12 deterministic split, and 100% test accuracy. Decision Tree reported 48 rows and 100% training accuracy. SVM reported 40 rows and 100% training accuracy. These values reproduce the designed laboratory datasets and are not claims of population performance.

## A.2 One-page run checklist

1. Install each project's declared dependencies before starting its server.

2. For KNN, Decision Tree, or SVM, rerun `python train_model.py` whenever its CSV or training code changes.
3. Serve browser-only `dist` folders over localhost so JSON and scripts load normally.
4. Wait for the page's ready indicator before entering a prediction.
5. Keep the units printed beside each field and interpret the evidence panel together with the main label.
6. Never use the health demonstrations for diagnosis or urgent decisions.

---

## Appendix B

---

# Glossary for a First-Time Reader

---

| Word                 | Plain meaning in this report                                                                                                                                             |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Algorithm            | A precise recipe of steps a computer follows.                                                                                                                            |
| Feature              | An input fact, such as height, glucose, or one pixel value.                                                                                                              |
| Target/label         | The example answer supplied during supervised learning.                                                                                                                  |
| Training             | Adjusting or constructing a model from examples; KNN mainly stores its labeled examples, while the other lab models learn parameters, rules, groups, or visual features. |
| Inference/prediction | Applying an existing model to a new input.                                                                                                                               |
| Parameter            | A learned or chosen number that affects behavior, such as a weight, threshold, neighbor count, centroid, or confidence cutoff.                                           |
| Epoch                | One complete pass through all training examples.                                                                                                                         |
| Accuracy             | Fraction of checked labels predicted correctly; its meaning depends completely on which rows were checked.                                                               |
| Overfitting          | Learning training details so narrowly that new-data performance becomes worse.                                                                                           |
| Standardization      | Subtracting a feature's mean and dividing by its standard deviation.                                                                                                     |
| Confidence           | A model score associated with a guess; it is not a guarantee.                                                                                                            |
| Centroid             | The coordinate-wise average position of a cluster.                                                                                                                       |
| JSON                 | A text structure of names, arrays, numbers, and objects shared with the browser.                                                                                         |
| API                  | A defined request/response doorway between pieces of software.                                                                                                           |

---

# Bibliography

---

- [1] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967, doi: 10.1109/TIT.1967.1053964.
- [2] Centers for Disease Control and Prevention, “Adult BMI calculator,” 2024. [Online]. Available: <https://www.cdc.gov/bmi/adult-calculator/>. [Accessed: Sep. 13, 2026].
- [3] American College of Cardiology, “2021 AHA/ACC chest pain guideline,” Oct. 2021. [Online]. Available: <https://www.acc.org/Guidelines/Guidelines/2021/10/28/12/34/Chest-Pain>. [Accessed: Sep. 13, 2026].
- [4] National Heart, Lung, and Blood Institute, “Heart attack diagnosis,” Mar. 2022. [Online]. Available: <https://www.nhlbi.nih.gov/health/heart-attack/diagnosis>. [Accessed: Sep. 13, 2026].
- [5] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.
- [6] National Institute of Diabetes and Digestive and Kidney Diseases, “Diabetes tests and diagnosis,” Dec. 2022. [Online]. Available: <https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis>. [Accessed: Sep. 13, 2026].
- [7] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995, doi: 10.1007/BF00994018.
- [8] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proc. 5th Berkeley Symp. Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297. [Online]. Available: <https://digicoll.lib.berkeley.edu/record/113015?v=pdf>
- [9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2016, pp. 779–788. [Online]. Available: [https://openaccess.thecvf.com/content\\_cvpr\\_2016/html/Redmon\\_You\\_Only\\_Look\\_CVPR\\_2016\\_paper.html](https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html)
- [10] Ultralytics, “Ultralytics YOLOv8,” [Online]. Available: <https://docs.ultralytics.com/models/yolov8/>. [Accessed: Sep. 13, 2026].

- [11] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Computer Vision–ECCV 2014*, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1\_48.
- [12] Pallets, “Uploading files,” *Flask Documentation*. [Online]. Available: <https://flask.palletsprojects.com/en/stable/patterns/fileuploads/>. [Accessed: Sep. 13, 2026].
- [13] Pillow Contributors, “Pillow documentation,” [Online]. Available: <https://pillow.readthedocs.io/>. [Accessed: Sep. 13, 2026].
- [14] Ultralytics, “Train mode,” *Ultralytics YOLO Documentation*. [Online]. Available: <https://docs.ultralytics.com/modes/train/>. [Accessed: Sep. 13, 2026].